AGENTOS - A HYBRID MULTI-AGENT RUNTIME ENVIRONMENT FOR AUTONOMOUS AI APPLICATIONS

A PROJECT REPORT

Submitted by

GOGUL A (2023115020)

MOHAN VISHNU KUMAR M (2023115026)

OVIYASHREE C J (2023115052)

KUDIMI GUNA SEKHAR (2023115126)

submitted to the Faculty of

COMPUTER SCIENCE AND INFORMATION ENGINEERING

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY

CHENNAI - 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **AgentOS - A Hybrid Multi-Agent Runtime Environment for Autonomous AI Applications** is the bona fide work of Gogul A (2023115020), Mohan Vishnu Kumar M (2023115026), Oviyashree C J (2023115052), and Kudimi Guna Sekhar (2023115126), who carried out the project work under my supervision. Certified further that, to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: Chennai
DATE: 29/10/2026

Dr. SELVI RAVINDRAN
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI 600025

COUNTERSIGNED

Dr. P. YOGESH
HEAD OF THE DEPARTMENT
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY
CHENNAI 600025

ABSTRACT

Large-language-model applications are evolving from single-turn chat interfaces into systems that plan, use tools, preserve context, and coordinate multiple specialised agents. However, application developers commonly rebuild scheduling, memory, event propagation, execution monitoring, and failure handling for each project. This report presents AgentOS, a hybrid multi-agent runtime environment that provides these operating-system-style services around autonomous AI applications. The implemented system combines a Next.js user interface, a FastAPI gateway, a Redis-backed event path, MongoDB task history, ChromaDB semantic retrieval, structured LLM planning, dependency-aware directed acyclic graph execution, semantic context paging, a global priority-aging scheduler, hybrid local/cloud model routing, token and concurrency admission control, and a sandboxed Python code runner with bounded self-healing retries. The frontend exposes the generated plan graph and live scheduler and execution activity through WebSocket updates. The report distinguishes implemented facilities from proposed components such as complete role-based access control and durable coroutine restoration after a kernel restart. Verification is based on repository compile checks, TypeScript checks, and focused smoke tests rather than unsubstantiated performance claims. AgentOS therefore provides a modular foundation for studying reliable, observable, and resource-aware multi-agent execution.

Keywords: multi-agent runtime, semantic planning, DAG execution, context paging, priority scheduling.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. Selvi Ravindran**, Associate Professor, Department of Information Science and Technology, Anna University, College of Engineering, Guindy, for her guidance, technical suggestions, and continuous encouragement throughout this work.

We thank the faculty members and staff of the Department of Information Science and Technology for providing the academic environment and facilities required to undertake this project. We also acknowledge our classmates, friends, and families for their support during the design, implementation, testing, and documentation of AgentOS.

Gogul A
Mohan Vishnu Kumar M
Oviyashree C J
Kudimi Guna Sekhar

TABLE OF CONTENTS

ABSTRACT iii
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF SYMBOLS AND ABBREVIATIONS xi

1 INTRODUCTION 1
1.1 BACKGROUND 1
1.2 MOTIVATION 1
1.3 PROBLEM STATEMENT 2
1.4 OBJECTIVES 2
1.5 SCOPE 3
1.6 CONTRIBUTIONS 3
1.7 REPORT ORGANIZATION 3

2 LITERATURE SURVEY 4
2.1 OVERVIEW 4
2.2 AGENT RUNTIMES AND AUTONOMOUS AGENTS 4
2.3 MEMORY AND CONTEXT MANAGEMENT 5
2.4 PLANNING, TOOL USE, AND REFLECTION 6
2.5 SCHEDULING, RESOURCES, AND RELIABILITY 7
2.6 SECURITY AND SANDBOXED EXECUTION 8
2.7 EVENT-DRIVEN INFRASTRUCTURE 8
2.8 RESEARCH GAP AND POSITIONING 9

3 SYSTEM DESIGN 10
3.1 DESIGN PRINCIPLES 10
3.2 OVERALL ARCHITECTURE 10
3.3 HYBRID LLM REASONING ENGINE 12
3.4 FRONTEND, GATEWAY, AND EVENT FLOW 12
3.5 PLANNER AND DAG EXECUTION 13
3.6 CONTEXT PAGING 15
3.7 GLOBAL SCHEDULER AND RESOURCE MANAGER 15
3.8 RESOURCE MANAGEMENT 16
3.9 TOOL MANAGEMENT 16

3.10 AGENT LIFECYCLE AND EVENTS 17
3.11 SANDBOXED CODE RUNNER 18
3.12 IMPLEMENTED AGENTOS MODULES 19

4 IMPLEMENTATION 21
4.1 IMPLEMENTATION ENVIRONMENT 21
4.2 SEMANTIC AGENT RETRIEVAL 21
4.3 PLANNER AND FALLBACK 22
4.4 HYBRID PROVIDER ROUTING 23
4.5 DAG VALIDATION 24
4.6 PRIORITY-AGING SCHEDULER 25
4.7 RESOURCE ADMISSION AND TOKEN BUDGETS 26
4.8 REDIS EVENTS AND BACKEND FORWARDING 27
4.9 CONTEXT PAGE-OUT AND PAGE-IN 27
4.10 SANDBOXED EXECUTION AND SELF-HEALING 28
4.11 FRONTEND OBSERVABILITY 28
4.12 CONFIGURATION AND VERIFICATION 28
4.13 AGENTOS ALGORITHMIC SPECIFICATION 29

5 RESULTS AND PERFORMANCE ANALYSIS 36
5.1 REPORTING PROTOCOL 36
5.2 CONTEXT-PAGING ENGINE 36
5.2.1 Evaluation protocol 37
5.2.2 Synthetic smoke evaluation 38
5.2.3 LoCoMo pilot evaluation 39
5.2.4 Graphical analysis 39
5.3 PLANNER AND TOOL SELECTION 41
5.4 AGENT SCHEDULER 43
5.5 INTEGRATED INTERPRETATION AND LIMITATIONS 45

6 CONCLUSION AND FUTURE WORK 47

DECLARATION ON THE USE OF AI TOOLS 49

APPENDIX 51

A AGENT CATALOGUE AND INTERNAL WORKFLOWS 51
A.1 Agent A1: System Information Agent 53

A.2 Agent A2: Date and Time Agent 54
A.3 Agent A3: Research Agent 55
A.4 Agent A4: File and Document Agent 56
A.5 Content Creator Agent 57
A.6 Email Agent 58
A.7 Weather Agent 59
A.8 Loan EMI Calculator Agent 60
A.9 Crypto PnL Agent 61
A.10 Code Runner Agent 62
A.11 Data Analysis Agent 63
A.12 Decision Support Agent 64
A.13 Summarizer Agent 65
A.14 Translator Agent 66
A.15 Checklist Agent 67
A.16 Agent Creator 68

B AI-ASSISTED PROMPTS 69
B.1 LITERATURE-SURVEY SUPPORT 69
B.2 LANGUAGE AND EDITORIAL SUPPORT 70
B.3 SYNTHETIC TEST-CASE SUPPORT 71

REFERENCES 72

LIST OF TABLES

3.1	Implemented AgentOS modules	20
4.1	Principal Redis channels and keys	27
4.2	Selected AgentOS configuration values	29
5.1	Synthetic context-paging evaluation	38
5.2	LoCoMo pilot context-retrieval evaluation	39
5.3	AgentOS tool-selection adapter results	41
5.4	Priority-aging scheduler versus FIFO under steady-state workload	43
5.5	Priority-aging scheduler versus FIFO under queue-contention workload	44

LIST OF FIGURES

3.1 Overall AgentOS system architecture 11
3.2 Frontend, FastAPI gateway, Redis channels, AI kernel, and agent-workforce flow 13
3.3 Planner, validation, and dependency-aware execution flow 14
3.4 Context page-out, vector-store retrieval, and semantic page-in flow 15
3.5 Global agent scheduler, lifecycle events, resource checks, and runtime feedback 16
3.6 Resource-manager decision flow supplied for the AgentOS design 17
3.7 Tool selection, permission checking, invocation, and result return 18
3.8 Agent lifecycle transitions 18
3.9 Sandboxed Python execution and bounded self-healing loop 19

4.1 Optimised semantic planner and DAG construction 23

5.1 Success and evidence-recall comparison for the synthetic and LoCoMo pilot evaluations 40
5.2 Similarity-threshold sensitivity for context retrieval 40
5.3 Overall outcomes for the 50-trial AgentOS tool-selection evaluation 42
5.4 End-to-end exact-match accuracy by task category 42
5.5 Measured scheduler throughput under steady-state and queue-contention workloads 44
5.6 High- and low-priority queue-wait P_{95} under priority aging and FIFO 45

A.1 AgentOS agent catalogue 52
A.2 A1 system-information logic 53
A.3 A2 date-and-time logic 54
A.4 A3 research and source-verification logic 55
A.5 A4 file-operation and deletion-confirmation logic 56

A.6 Content Creator drafting logic 57
A.7 Email delivery and simulation-mode logic 58
A.8 Weather retrieval and reporting logic 59
A.9 Loan EMI and amortisation logic 60
A.10 Cryptocurrency PnL and ROI logic 61
A.11 Code generation, sandbox execution, and repair logic 62
A.12 Data-analysis inspection and calculation logic 63
A.13 Decision-support synthesis logic 64
A.14 Summarization and fact-preservation logic 65
A.15 Translation and tone-preserving transformation logic 66
A.16 Checklist construction logic 67
A.17 Dynamic agent creation and registration logic 68

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
DAG	Directed Acyclic Graph
FSM	Finite-State Machine
IPC	Inter-Process Communication
LLM	Large Language Model
MCP	Model Context Protocol
MongoDB	Document-oriented database used for task history and metadata
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
Redis	In-memory data store used for queues and event transport
UI	User Interface
WebSocket	Bidirectional communication protocol over a persistent connection
α	Aging factor used in effective-priority calculation
$P_{\max}$	Maximum effective priority
T_q	Cooperative dispatch time quantum

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Large-language-model (LLM) applications have progressed from reactive question-answering interfaces to systems that can plan, call tools, maintain memory, and delegate work to specialised agents. A useful multi-agent application therefore needs more than a prompt and a model endpoint. It needs a runtime that can represent work, enforce dependencies, manage limited provider and execution resources, preserve useful context, report progress, and recover safely from failures.

Operating-system concepts provide a useful design analogy. A task is submitted to a gateway, a planner decomposes it into executable steps, a scheduler selects runnable agent work, a resource manager performs admission control, and an event path exposes state transitions. AgentOS applies this analogy to LLM-powered agents while retaining the uncertainty and latency of model calls.

1.2 MOTIVATION

In a conventional chatbot, application logic, conversation state, tool invocation, and error handling are often tightly coupled. This makes it difficult to run independent work concurrently or to explain why an agent was selected. Long conversations also exceed the practical context window of a model, while blindly truncating history can remove information needed by later steps. Finally, unrestricted execution services and tools create safety and observability risks.

AgentOS is motivated by the need for a reusable runtime layer. The project investigates how planning, agent retrieval, DAG execution, semantic context paging, global scheduling, resource accounting, event streaming, and sandboxed execution can be composed into one observable system.

1.3 PROBLEM STATEMENT

There is no single lightweight runtime in the project environment that coordinates heterogeneous agents across multiple concurrent tasks while enforcing dependencies, observing token and concurrency budgets, preserving relevant context, and exposing execution state to a user interface. Reimplementing these facilities inside each application increases duplication, makes failures difficult to diagnose, and prevents systematic evaluation of scheduling and resource policies.

1.4 OBJECTIVES

The objectives of AgentOS are:

1. Develop a modular AI-kernel-style runtime for specialised agents;
2. Generate structured execution plans, retrieve suitable agents semantically, and validate their dependencies as a DAG;
3. Manage bounded conversational context through semantic page-out and page-in using a vector-backed swap space;
4. Schedule runnable work with priority aging, fairness, cooperative dispatch, concurrency limits, and token budgets; and
5. Execute agent tasks reliably while publishing lifecycle events and measurable activity to the user interface.

1.5 SCOPE

The current phase covers a Next.js frontend, FastAPI gateway, Python kernel, Redis event transport, MongoDB task persistence, ChromaDB retrieval, structured planning, DAG validation and Mermaid generation, context paging, a global in-process scheduler, resource admission, hybrid local/cloud model routing, and a registry of specialised agents for information, research, documents, communication, calculation, and bounded execution tasks. The scope does not claim complete RBAC, distributed scheduler workers, forced termination of active LLM calls, or automatic coroutine restoration. These are documented as future work.

1.6 CONTRIBUTIONS

The project contributions are a unified execution model for heterogeneous agents, a visible plan graph, a Redis-mediated activity stream, a semantic context paging mechanism, a priority-aging scheduler with resource admission, and a bounded execution and recovery path for specialised agents. The implementation also makes a deliberate distinction between design aspirations and features that are present and testable in the repository.

1.7 REPORT ORGANIZATION

Chapter 2 reviews related work on autonomous agents, memory, planning, scheduling, tool use, security, event transport, and vector retrieval. Chapter 3 presents the AgentOS architecture and data flows, while Chapter 4 describes its algorithms, storage, events, and verification evidence. Chapter 5 reports the evaluation protocol and results for context paging, planner tool selection, and agent scheduling. The conclusion synthesises the findings and future work; the appendix documents the registered agents and internal workflows.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

This chapter reviews work that informs the design of AgentOS. The survey combines peer-reviewed or archival research with official technical documentation for infrastructure components. Each group is considered in terms of its contribution, limitation for the present project, and the way AgentOS uses or extends the idea.

2.2 AGENT RUNTIMES AND AUTONOMOUS AGENTS

AIOS frames an LLM agent operating system around scheduling, context management, memory, storage, access control, and tool management [1]. Its operating-system analogy directly motivates the AgentOS kernel, although the current AgentOS implementation is smaller and uses a Python/Redis/MongoDB stack rather than a complete OS abstraction. The broader survey by Wang et al. classifies LLM-based agents by construction, applications, and evaluation [2]; it identifies coordination and reliability as open challenges that AgentOS addresses through explicit events and DAG dependencies.

The LLM-as-an-OS proposal treats models as system resources and agents as applications [3]. AutoGen demonstrates programmable multi-agent conversation and human-in-the-loop patterns [4], while LangGraph provides graph-based orchestration and durable execution concepts [5]. These systems

show the value of explicit state and orchestration, but do not provide the exact global priority-aging and token-admission policy implemented here.

AgentBench provides a complementary evaluation perspective by testing language models as agents in eight interactive environments rather than treating generation quality as the only outcome [6]. Its multi-turn, open-ended tasks expose failures in long-horizon reasoning, decision making, and instruction following. This distinction is important for AgentOS: a plan may be syntactically valid while the resulting tool calls, dependencies, or execution state are still incorrect. AgentOS therefore evaluates planning and runtime behaviour separately instead of presenting a single language-generation score as evidence of system reliability.

2.3 MEMORY AND CONTEXT MANAGEMENT

MemGPT introduces virtual context management that moves information between active context and external storage [7]. MemoryOS organises memory into short-, mid-, and long-term layers and evaluates memory retention for conversational agents [8]. These works motivate AgentOS context page-out and page-in. AgentOS uses semantic embeddings in ChromaDB and an active-window policy; it currently provides a project-level paging mechanism rather than a learned memory hierarchy or an independently benchmarked memory score.

Recent evaluation work also shows why memory retrieval should not be reduced to isolated fact lookup. LongMemEval measures information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention in long-term interactive conversations [9]. These dimensions expose failure modes that a single retrieval-recall value can hide, particularly when previously stored information becomes outdated or when the correct response is

to abstain. The present AgentOS evaluation is deliberately narrower: it measures evidence visibility and retrieval metrics for the paging mechanism, while answer-level judging and dynamic knowledge-update evaluation remain outside the current scope.

Retrieval-Augmented Generation (RAG) combines generation with a non-parametric knowledge store [10]. ChromaDB documents collections, embeddings, and distance-based retrieval [11]. AgentOS uses this pattern both for agent capability retrieval and for swapped context. The limitation is that retrieval quality depends on the embedding model and distance configuration; the current implementation exposes similarity metadata but does not claim domain-wide retrieval benchmarks.

Recent evaluation frameworks further distinguish retrieval quality from the quality of the final generated response. RAGAs evaluates retrieval relevance, contextual faithfulness, and answer quality without requiring a complete set of human reference answers [12]. ARES similarly separates context relevance, answer faithfulness, and answer relevance, using synthetic training data together with a small human-annotated set for calibration [13]. These studies provide a useful direction for extending AgentOS: the present context-paging experiments report evidence visibility and retrieval recall, while future evaluations could add answer-level faithfulness and relevance measures without conflating them with page-in success.

2.4 PLANNING, TOOL USE, AND REFLECTION

ReAct interleaves reasoning and action so that an agent can use external tools during problem solving [14]. Toolformer studies self-supervised learning for deciding when and how to call APIs [15]. Generative Agents combine memory, reflection, and planning for believable behaviour [16]. Reflexion

uses verbal feedback to improve subsequent attempts without updating model weights [17]. AgentOS adopts structured planning, explicit tool boundaries, persisted events, and bounded self-healing, but does not train models or claim the autonomous reflection capabilities of these research systems.

ToolLLM and its ToolBench dataset study tool use at a substantially larger scale, covering more than 16,000 real-world REST APIs across 49 categories and evaluating both API recommendation and multi-step call paths [18]. This work highlights that tool use depends on capability retrieval, argument construction, and execution-path selection, not only on the model's general reasoning ability. AgentOS addresses the same separation at a smaller system level through a registered agent catalogue, semantic candidate retrieval, structured plans, and an end-to-end tool-selection evaluation. The comparison is conceptual rather than numerical: AgentOS does not claim ToolBench-scale coverage or model-training results.

Recent survey work provides a useful vocabulary for positioning this architecture within workflow-optimisation research. Yue et al. distinguish static workflow templates from dynamic runtime graphs and separate reusable designs, run-specific realised graphs, and execution traces [19]. They also argue that workflow evaluation should consider graph structure, execution cost, robustness, and variation across inputs in addition to downstream task metrics. This perspective supports AgentOS's separation of planning, DAG validation, scheduling, and runtime evidence, while also identifying graph-level optimisation and cost-aware evaluation as directions for future work.

The Transformer architecture remains the dominant basis for modern LLMs [20]. Its context limitation makes external context management relevant to AgentOS, while the model itself remains an external provider selected by the planner/runtime.

2.5 SCHEDULING, RESOURCES, AND RELIABILITY

Classical operating-system scheduling shows why priority alone can starve old work and why aging can improve fairness [21]. Linux cgroups provide a practical model for resource accounting and isolation [22]. AgentOS adapts these ideas to agent work: effective priority increases with queue wait, admission checks global and per-task slots, and token reservations are tracked before dispatch. Unlike a CPU scheduler, AgentOS cannot forcibly preempt an in-flight remote LLM request; its dispatch quantum is cooperative, with full execution checkpoints remaining a future extension.

The workflow and orchestration literature emphasises explicit dependency graphs and stateful execution. LangGraph persistence documents checkpointed graph state [23], while Temporal documents durable workflow execution [24]. AgentOS currently validates and executes a DAG in-process and persists task/event evidence; automatic continuation of interrupted coroutines remains future work.

2.6 SECURITY AND SANDBOXED EXECUTION

The Model Context Protocol specifies a standard way for AI applications to connect models with tools and data sources [25]. OWASP's LLM security guidance highlights risks including excessive agency, insecure tool use, and code execution [26]. Python subprocess execution and filesystem scoping are therefore bounded in the code runner. Firecracker demonstrates the stronger isolation achievable with microVMs [27]. AgentOS's current runner limits duration and output and scopes its working directory, but it is not a substitute for a hardened kernel or microVM sandbox.

2.7 EVENT-DRIVEN INFRASTRUCTURE

Redis documents Pub/Sub as a low-latency message distribution mechanism [28]. MongoDB provides durable document storage suitable for task history and metadata [29]. AgentOS uses Redis for transient task/event transport and queue metadata and MongoDB for durable task records. Redis Pub/Sub alone is not a durable event log; recovery therefore persists relevant state in Redis hashes and MongoDB events, with full replay and exactly-once semantics left for future work.

2.8 RESEARCH GAP AND POSITIONING

The reviewed systems provide strong solutions for individual concerns: planning, memory, graph orchestration, tool use, security, or workflow durability. AgentOS's contribution is an integrated educational/runtime prototype that makes these concerns observable together in one project. Its distinctive implementation choices are semantic agent selection, explicit DAG validation, global priority-aging scheduling, token/concurrency admission, Redis lifecycle events, context paging, and frontend graph/activity observability. The current phase prioritises a transparent and testable architecture over claims of production isolation or benchmark leadership.

CHAPTER 3

SYSTEM DESIGN

3.1 DESIGN PRINCIPLES

AgentOS is designed as a modular runtime. The gateway handles external requests, the orchestrator converts a prompt into a structured plan, and the kernel owns execution concerns. Redis is used for low-latency coordination and event transport, while MongoDB stores task history. ChromaDB stores embeddings used for semantic agent retrieval and context swap. The design keeps the current dependency truth in the DAG executor and treats scheduler decisions as admission and dispatch decisions.

3.2 OVERALL ARCHITECTURE

As shown in Figure 3.1, AgentOS is organised as an overall runtime architecture connecting the user interface, gateway, planner, kernel, storage, tools, agents, and model engines. The figure combines implemented runtime components with architectural modules that are planned or only partially implemented; their implementation status is documented separately in Section 3.12. A user submits a task through the Next.js interface. The FastAPI gateway creates or retrieves task state and communicates with the kernel. The planner selects agents and creates steps; the kernel validates and executes those steps while publishing activity events.

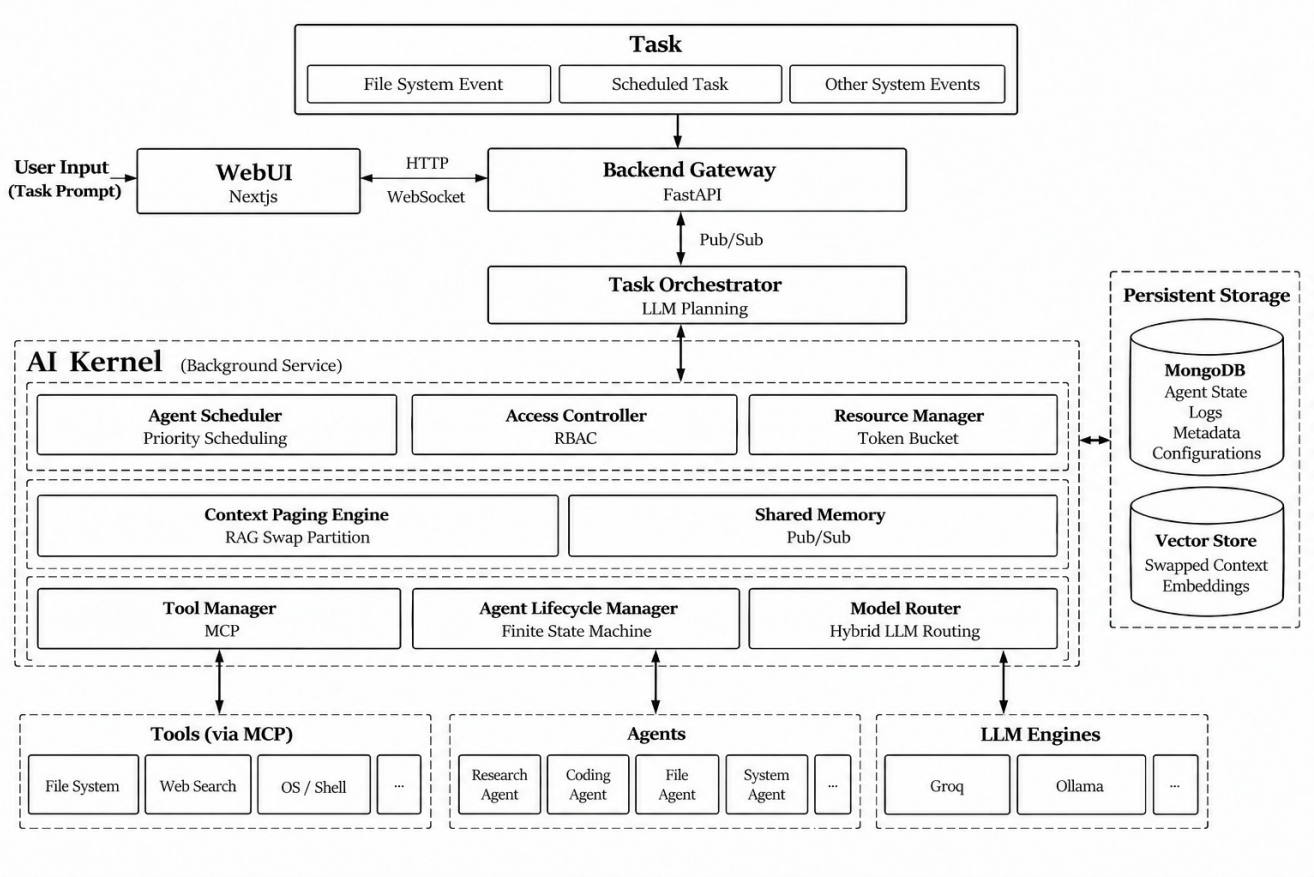


Figure 3.1: Overall AgentOS system architecture

3.3 HYBRID LLM REASONING ENGINE

AgentOS uses a privacy-aware hybrid reasoning engine to balance cloud-model capability with local data protection. The model factory exposes one LangChain-compatible facade to the planner, agents, and response generator, while the facade selects a provider for each invocation. Groq is used for non-sensitive orchestration work when a cloud credential is configured; Ollama is used for privacy-sensitive work such as local file inspection, repository or code analysis, academic material, personal information, credentials, and other locally scoped data. The local path is configured with an Ollama model such as `llama3.2` or `qwen3:8b` and communicates with the host-local Ollama service.

The router applies conservative input classification before a model call. It extracts text from strings, message lists, and structured inputs, then checks privacy and local-resource indicators. A sensitive classification has precedence over cloud routing, so the request remains local even when Groq is available. The router also supports explicit `local`, `cloud`, and `hybrid` modes. In hybrid mode, an unavailable or failed Groq request is retried through Ollama, providing a local continuity path. This design makes the provider choice a kernel concern rather than a frontend concern: the UI reports the decision emitted by the kernel instead of attempting to infer privacy from the prompt.

The same facade preserves structured-output and tool-binding interfaces. Consequently, planning, agent tool calls, and final response generation can all use the routing policy without requiring separate graph implementations. Each selected route produces an `llm.ROUTED` event containing the provider, sensitivity flag, reason, and operation. These events are persisted with the task and forwarded to the frontend, making the privacy boundary auditable during execution.

3.4 FRONTEND, GATEWAY, AND EVENT FLOW

The frontend sends task prompts over HTTP and receives live activity over WebSocket. The backend listener subscribes to the kernel event channel and forwards events to connected clients. Figure 3.2 shows the separation between the task submission channel and the kernel event channel. MongoDB persistence is performed by the kernel/repository layer so the UI can later query task history.

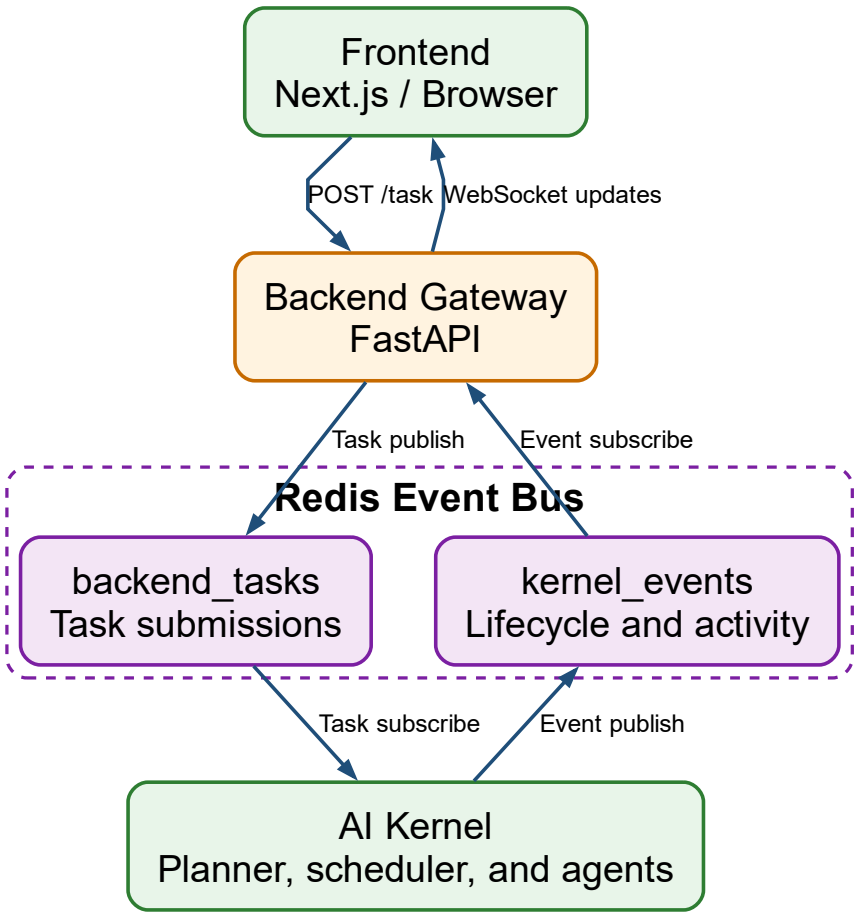


Figure 3.2: Frontend, FastAPI gateway, Redis channels, AI kernel, and agent-workforce flow

3.5 PLANNER AND DAG EXECUTION

The planner retrieves relevant agent descriptions, requests a structured execution plan, and applies a fallback plan when the provider is unavailable. Each step includes an identifier, agent name, description, and dependency list. The validator canonicalises identifiers, rejects unknown dependencies and cycles, and supplies a sequential fallback only when a plan omits dependencies. The executor runs only steps whose prerequisites are complete. Figure 3.3 summarises this path. The frontend uses the same validated plan to generate Mermaid source and draw the plan graph, ensuring that displayed edges represent dependencies rather than decorative links.

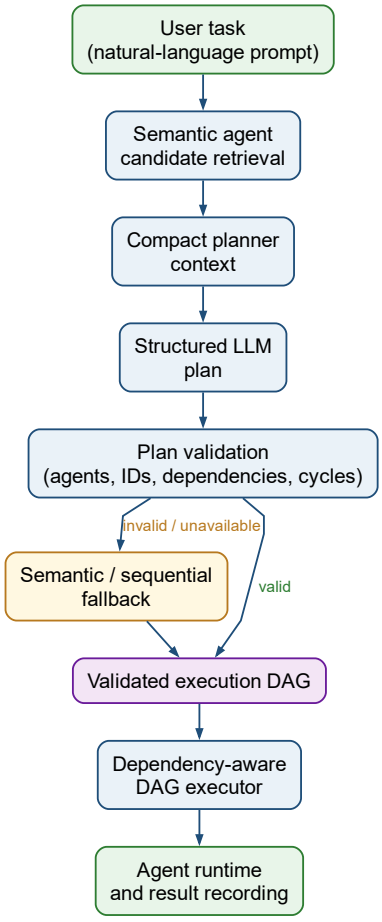


Figure 3.3: Planner, validation, and dependency-aware execution flow

3.6 CONTEXT PAGING

The context pager keeps a bounded active message window. When the window is exceeded, older or less relevant messages are embedded and stored in the ChromaDB swap collection. A later query retrieves semantically related pages and injects them into the active context. ChromaDB's cosine space returns distance; the implementation converts it as $s = 1 - d$, where d is the returned cosine distance. Figure 3.4 presents the page-out and page-in flow. This mechanism reduces active prompt growth but does not guarantee perfect recall.

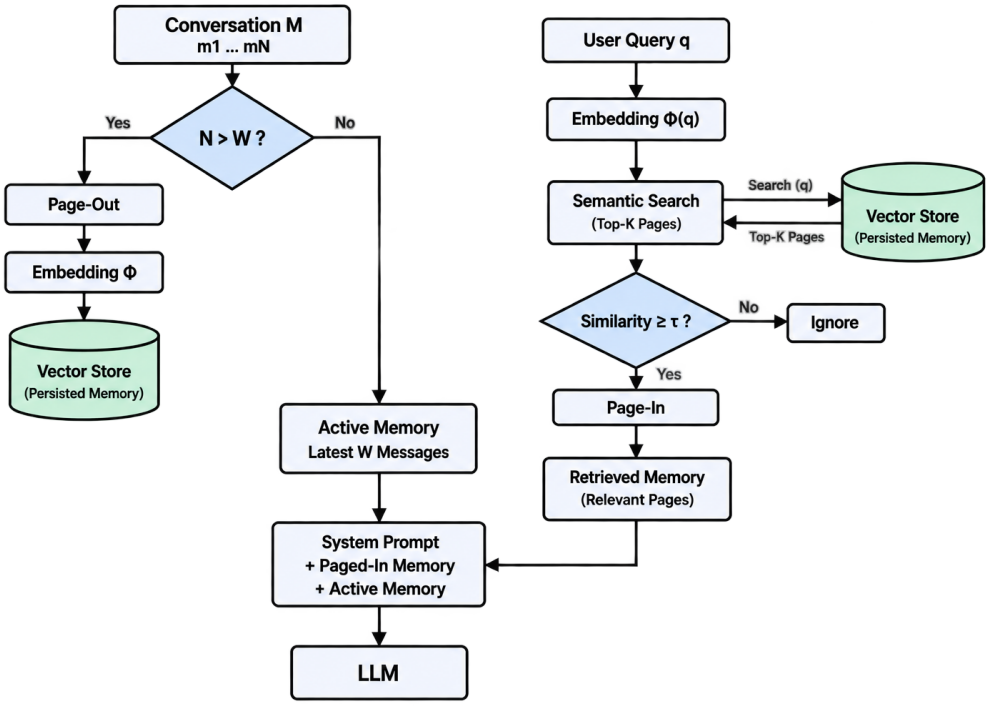


Figure 3.4: Context page-out, vector-store retrieval, and semantic page-in flow

3.7 GLOBAL SCHEDULER AND RESOURCE MANAGER

The scheduler maintains ready records and selects work globally across active tasks. Effective priority is computed from base priority and queue

waiting time, bounded by a maximum. Before dispatch, the resource manager checks the global slot limit, per-task slot limit, and token budget. The current defaults are three global execution slots, two slots per task, and a configurable 50,000-token task budget. Dispatch is cooperative: a running remote LLM request is not forcibly killed. Figure 3.5 shows the decision path.

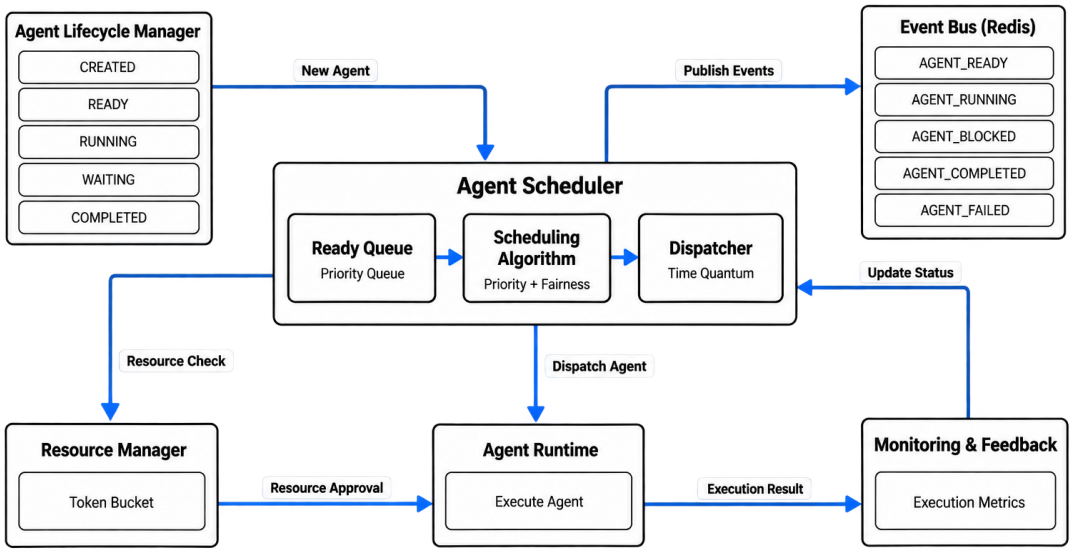


Figure 3.5: Global agent scheduler, lifecycle events, resource checks, and runtime feedback

3.8 RESOURCE MANAGEMENT

The resource-manager design in Figure 3.6 expands the admission point in the scheduler. It separates token-bucket accounting, quota monitoring, process monitoring, resource checking, and the final decision. In the current implementation, the corresponding resource manager enforces global and per-task execution slots and a token budget; the policy store, process monitor, and hard preemption shown in the figure are design extensions.

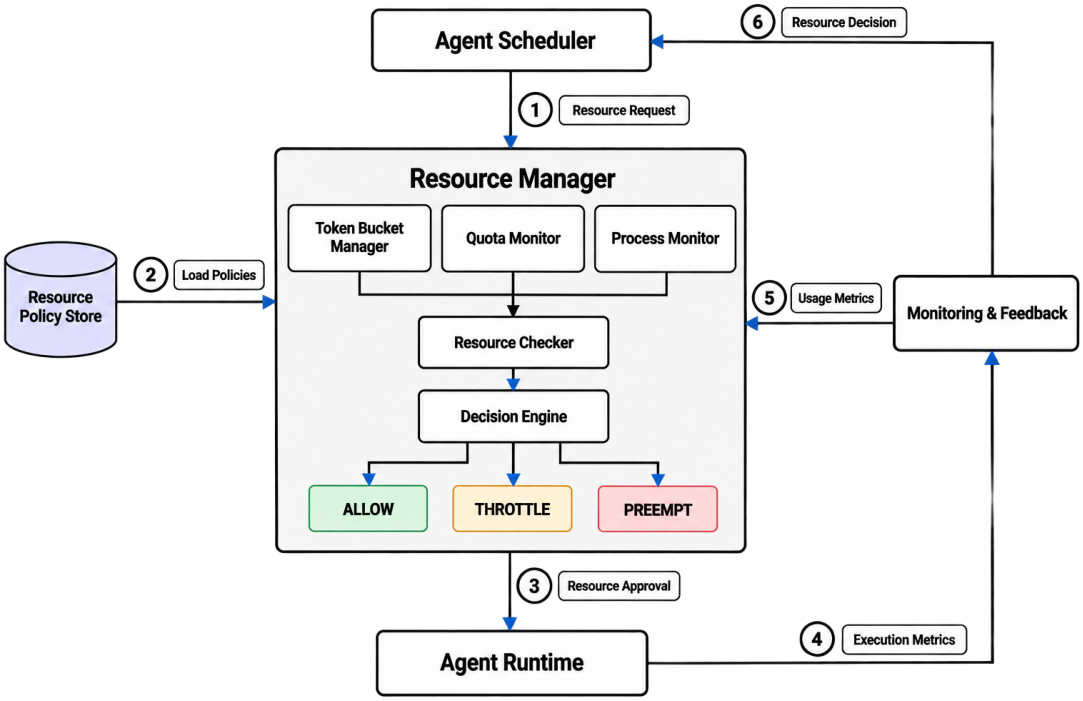


Figure 3.6: Resource-manager decision flow supplied for the AgentOS design

3.9 TOOL MANAGEMENT

Figure 3.7 shows the intended tool-manager boundary between an agent, a tool registry, permission policy, and local or external services. AgentOS currently exposes bounded tools through the runtime and code runner; a complete permission store and MCP-compatible tool manager remain proposed components.

3.10 AGENT LIFECYCLE AND EVENTS

An agent step progresses from CREATED to READY, RUNNING, and COMPLETED. It can move to WAITING when a resource or dependency is unavailable, or to BLOCKED/FAILED when execution cannot proceed. The scheduler and runtime publish structured lifecycle events containing task, step, agent, execution, status, priority, queue wait, and attempt information. Fig-

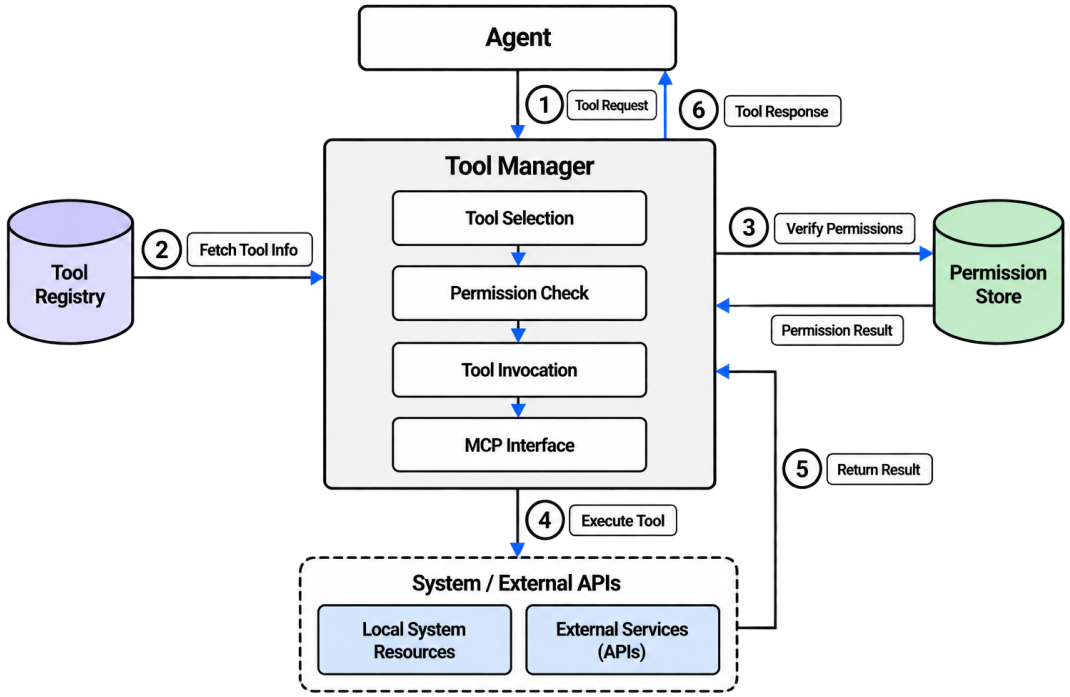


Figure 3.7: Tool selection, permission checking, invocation, and result return

ure 3.8 shows the state model. Complete RBAC and comprehensive state-machine enforcement remain extensions beyond the current implementation boundary.

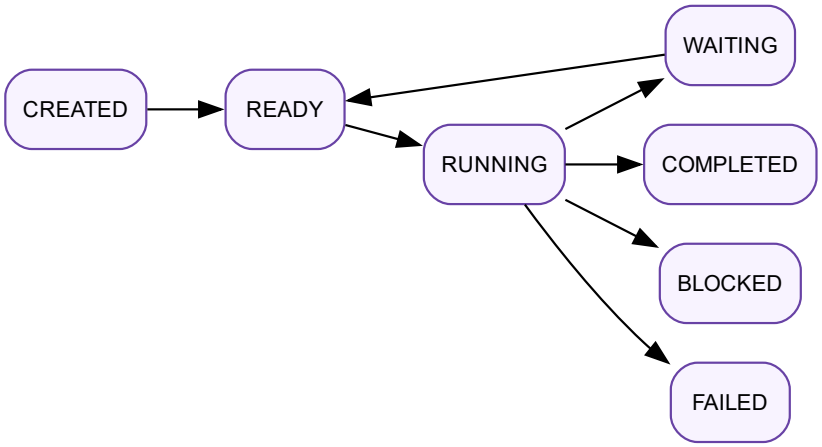


Figure 3.8: Agent lifecycle transitions

3.11 SANDBOXED CODE RUNNER

The code runner receives generated Python, writes it into a scoped working directory, and invokes it through an asynchronous subprocess. It bounds execution time and captured output. A non-zero exit code or traceback is returned to the coder/tester agent, which may produce a corrected version for at most three attempts. Output lines can be published as kernel events for live frontend observation. Figure 3.9 shows this flow. The current isolation is process and directory scoping; hardened container or microVM isolation is future work.

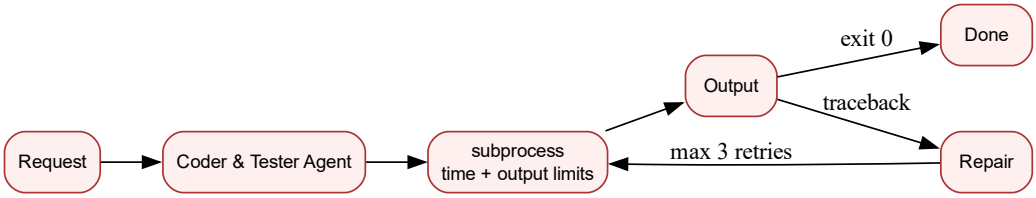


Figure 3.9: Sandboxed Python execution and bounded self-healing loop

3.12 IMPLEMENTED AGENTOS MODULES

Table 3.1 summarises the principal modules implemented and exercised in the current AgentOS runtime.

Table 3.1: Implemented AgentOS modules

Module	Principal responsibility	Status
Frontend and gateway	User interface, task API, and live activity stream	Implemented
Planner and DAG execution	Structured plans, agent selection, and dependency validation	Implemented
Hybrid model router	Local/cloud model selection and provider fallback	Implemented
Agent and tool registry	Capability descriptions, function schemas, and candidate retrieval	Implemented
Context paging	ChromaDB swap, semantic retrieval, and bounded context	Implemented
Scheduler and resource admission	Priority aging, concurrency limits, and token budgets	Implemented; cooperative dispatch
Persistence and events	MongoDB task history and Redis lifecycle transport	Implemented
Code runner	Bounded subprocess execution and repair retries	Implemented; not microVM isolation
Observability	Lifecycle, queue, priority, and resource activity	Implemented

CHAPTER 4

IMPLEMENTATION

4.1 IMPLEMENTATION ENVIRONMENT

The kernel is implemented in Python using asynchronous services. The backend is FastAPI-based and the frontend is implemented with Next.js and TypeScript. Redis provides task/event channels and scheduler metadata; MongoDB stores task records and event history; ChromaDB provides persistent vector collections for planning and context swap. The LLM layer is provider-independent at the graph boundary: a hybrid model facade presents one asynchronous LangChain interface while selecting Groq or Ollama per invocation. Provider-specific LLM usage is treated as optional metadata because some providers do not return reliable token counts.

4.2 SEMANTIC AGENT RETRIEVAL

Agent descriptions are embedded in a persistent ChromaDB collection. A task description is used as a query, and the top candidate records are returned with their distances and metadata. In the configured cosine space, ChromaDB returns $d = 1 - \cos(\theta)$, so the implementation reports $s = 1 - d$. It does not divide by two; that transformation would be appropriate for a different distance normalisation. An LRU cache reduces repeated retrieval for identical descriptions. Algorithm 4.1 summarises this candidate-retrieval and cosine-distance conversion procedure.

Algorithm 4.1 Semantic Agent Retrieval

Require: task description q, agent collection C, result count k
1: $r \leftarrow C.\text{query}(q,k)$
2: **for all** candidate distances d_i in r **do**
3: $s_i \leftarrow 1 - d_i$
4: attach $(agent_i, s_i, description_i)$ to candidate
5: **end for**
6: sort candidates by s_i descending
7: **return** top candidates

4.3 PLANNER AND FALLBACK

The planner asks the configured LLM for a structured execution plan. The response is parsed into steps and normalised against the agent registry. If the LLM request fails, exceeds quota, or produces invalid structure, a deterministic fallback creates a usable plan from the best retrieved agent(s). This maintains task execution and observability without pretending that the fallback has the same reasoning quality as the provider.

Figure 4.1 illustrates the complete optimised planning path, from semantic candidate retrieval through structured plan validation, fallback routing, DAG construction, scheduling, and runtime execution.

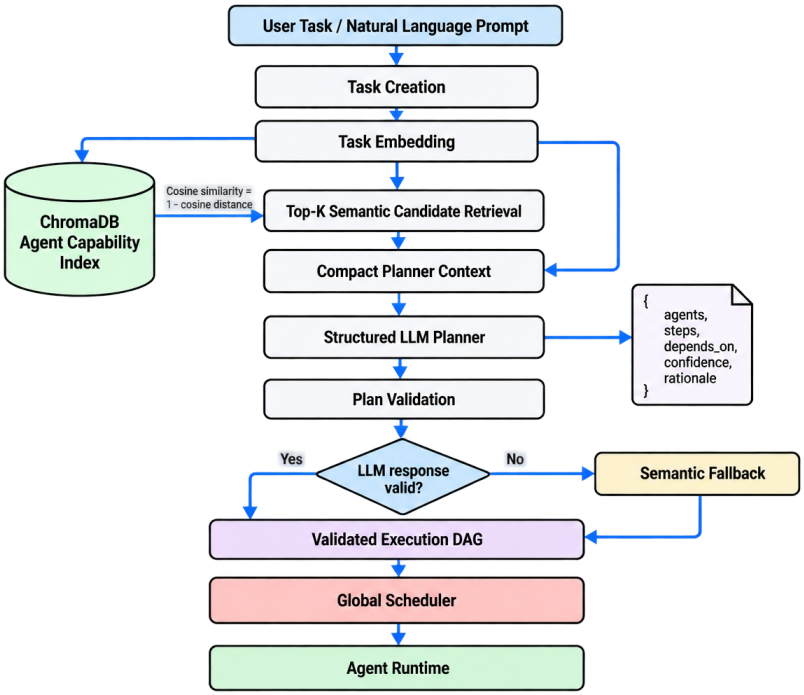


Figure 4.1: Optimised semantic planner and DAG construction

4.4 HYBRID PROVIDER ROUTING

The hybrid model implementation is centred in `kernel/models/model.py`. `HybridChatModel` wraps a `ChatGroq` instance when `GROQ_API_KEY` is available and a `ChatOllama` instance configured with `OLLAMA_MODEL` and `OLLAMA_BASE_URL`. The default mode is `hybrid`; `local` forces all calls to Ollama, while `cloud` permits Groq for non-sensitive calls. Sensitive input always overrides cloud selection. The Ollama client uses a configurable request timeout, bounded output length, and a short keep-alive interval so local failures do not remain indefinitely pending.

Sensitivity is currently implemented as a conservative deterministic classifier rather than a second LLM call. It examines the text extracted from the invocation and matches indicators for personal or confidential data, academic

work, local paths and files, repositories and code, credentials, and medical or financial information. This avoids sending the classification question itself to a cloud provider. When a Groq call is selected but fails, the facade retries the same operation with Ollama. Structured-output and tool-bound runnables are created through the facade, so the policy is applied consistently to planner JSON, agent tool calls, and final responses.

Algorithm 4.2 Privacy-Aware Hybrid LLM Routing

Require: invocation input x, routing mode m, local model L, optional cloud model G
Ensure: provider response y

$t \leftarrow \textsc{ExtractText}(x)$
$s \leftarrow \textsc{SensitiveMatch}(t)$
if $m = \texttt{local}$ **or** $s = \textsc{true}$ **then**
    $p \leftarrow L$
else if G exists and $m \in \{\texttt{hybrid}, \texttt{cloud}\}$ **then**
    $p \leftarrow G$
else
    $p \leftarrow L$
end if
emit `llm.ROUTED` with provider, sensitivity, reason, and operation
$y \leftarrow \textsc{Invoke}(p, x)$
if $p = G$ and invocation fails **then**
    $y \leftarrow \textsc{Invoke}(L, x)$
end if
return y

4.5 DAG VALIDATION

Every step has a unique identifier and a list of prerequisite identifiers. Validation rejects duplicate identifiers, unknown agents, unknown dependencies, and cycles. A Kahn-style topological pass counts nodes removed from the ready set. If the count is smaller than the number of steps, the dependency graph contains a cycle and is rejected. The same validated dependency lists are used to generate Mermaid edges. Algorithm 4.3 specifies the cycle-detection procedure.

Algorithm 4.3 Dependency Validation by Cycle Detection

Require: step set S
build adjacency list and indegree for every step
$Q \leftarrow$ all steps with indegree zero
$visited \leftarrow 0$
while Q is not empty **do**
    remove a step u from Q; $visited \leftarrow visited + 1$
    for all successors v of u **do**
        decrement indegree(v)
        if indegree(v) $= 0$ **then** add v to Q
        end if
    end for
end while
if $visited \neq |S|$ **then return** invalid cyclic plan
elsereturn valid plan
end if

4.6 PRIORITY-AGING SCHEDULER

The scheduler stores ready records in a Redis sorted set and keeps execution metadata in hashes. The selection score is conceptually computed as:

$$P_{effective} = \min(P_{base} + \alpha W, P_{max}), \tag{4.1}$$

where W is queue waiting time. The implementation uses an in-process pending set for dispatch coordination and persists the record so that ready and waiting work is visible and recoverable. A selected record is marked RUNNING, dispatched once, and then completed, requeued, or failed according to the result. Algorithm 4.4 gives the corresponding cooperative scheduling procedure.

Algorithm 4.4 Global Priority-Aging Cooperative Scheduler

while scheduler is active **do**
    receive and persist READY records
    if ready queue is empty **then** wait for event; continue
    end if
    for all ready records a_i **do** calculate $P_{effective,i}$ using Equation 4.1
    end for
    $a^* \leftarrow \arg\max P_{effective,i}$
    if resource manager denies a^* temporarily **then** mark WAITING and requeue
    else if resource manager denies permanently **then** mark FAILED and publish event
    elsedispatch a^* cooperatively and publish RUNNING
        if execution completes **then** publish COMPLETED
        else if execution waits or is requeued **then** publish WAITING or PREEMPTED
        elsepublish FAILED
        end if
    end if
    release resource reservations and record metrics
end while

4.7 RESOURCE ADMISSION AND TOKEN BUDGETS

The resource manager checks active global slots, active slots for the task, and the cumulative estimated token reservation for the task. A temporary denial leaves work waiting; an exhausted task budget is converted into a blocked/failed outcome rather than leaving the task silently pending. Actual provider usage, when available, is recorded separately from estimated admission tokens. This distinction explains why a provider such as Groq may display zero usage metadata even while the scheduler correctly enforces its configured estimate.

4.8 REDIS EVENTS AND BACKEND FORWARDING

Events are JSON records containing task identifier, step identifier, agent name, execution identifier, status, timestamp, priority, effective priority, queue wait, and attempt. The kernel publishes them on `kernel_events`; the backend subscribes and forwards them over WebSocket. Scheduler records use keys such as `agentos:scheduler:ready` and `agentos:scheduler:job:<execution_id>`. MongoDB persists task history and event records for later inspection. Table 4.1 summarises the Redis channels and scheduler keys used for task transport, lifecycle events, queue persistence, and recovery.

Table 4.1: Principal Redis channels and keys

Name	Purpose	Durability role
`backend_tasks`	New task/submission messages	Transport
`kernel_events`	Lifecycle, scheduler, and execution events	Transport plus Mongo persistence
`agentos:scheduler:ready`	Ready queue sorted-set metadata	Recovery aid
`agentos:scheduler:job:<id>`	Per-execution state hash	Recovery aid
`agentos:task:<id>`	Task-level scheduler state	Inspection/recovery

4.9 CONTEXT PAGE-OUT AND PAGE-IN

The pager moves messages from the active conversation state to a persistent ChromaDB collection after a configured window is exceeded. Each page stores message content, task metadata, and an embedding. On a later turn, the current query is embedded and the collection is searched; selected messages are reintroduced in relevance order. The implementation emits page-out and page-in metrics, allowing the UI or logs to verify that paging occurred.

4.10 SANDBOXED EXECUTION AND SELF-HEALING

The code runner creates a scoped temporary script, launches it with `asyncio.create_subprocess_exec`, captures stdout/stderr, and terminates the subprocess after a strict timeout. Output size is limited to prevent unbounded memory use. On a traceback, the error is supplied to the coder/tester agent and a revised script is attempted, up to three times. The runner publishes line-oriented output and final status as kernel events. No claim is made that an ordinary subprocess is equivalent to a container or microVM security boundary.

4.11 FRONTEND OBSERVABILITY

The task activity panel displays lifecycle events, scheduler state, queue position, effective priority, waiting time, attempt number, resource decisions, and token totals when provider metadata is present. It also displays the latest hybrid routing decision as `Groq / cloud` or `Ollama / local`, including a privacy-sensitive message when applicable. The `llm.ROUTED` events appear in the live timeline and are persisted with task history, so the provider selected for planning, agent execution, and response generation can be inspected after completion. The plan graph modal displays the structured graph and Mermaid source. Token fields distinguish input, output, total, and configured budget; when the provider omits usage, the interface reports that the metadata is unavailable rather than inventing a count.

4.12 CONFIGURATION AND VERIFICATION

Table 4.2 lists the important policy values. Verification for this phase consists of Python compilation, TypeScript type checking, scheduler/resource smoke tests, DAG validation checks, the seeded priority-versus-FIFO scheduler

workload benchmark reported in Chapter 5, and manual event/graph inspection. The benchmark is a same-host scheduler comparison with stubbed persistence; it is not a production capacity or model-quality claim.

Table 4.2: Selected AgentOS configuration values

Configuration	Default	Role
Global execution slots	3	Maximum simultaneously admitted jobs
Per-task execution slots	2	Fairness/isolation across tasks
Task token budget	50,000	Cumulative estimated admission limit
Maximum scheduler priority	configurable	Upper bound for aging
Context active window	configurable	Number of active messages
Self-healing attempts	3	Maximum code repair retries
LLM routing mode	`hybrid`	`hybrid`, `local`, or `cloud` provider policy
Ollama model	`llama3.2`	Local model used for sensitive input and fallback
Ollama endpoint	`localhost`	Host-local Ollama service at port 11434
Ollama output limit	512 tokens	Maximum generated tokens per local call
Ollama timeout	120 seconds	Maximum local provider request duration

4.13 AGENTOS ALGORITHMIC SPECIFICATION

The following algorithm sheets define the AgentOS execution design. Together they specify the relationship between the planner, executor, lifecycle manager, context pager, dependency-aware DAG executor, scheduler, and priority-aging model. They complement the executable implementation described above, but should be read as algorithmic specifications rather than as evidence that every operation is fully realised in the current runtime. In particular, the sheets that describe full execution-context restoration, hard time-slice suspension, permission stores, or preemption represent planned extensions where

the current implementation remains cooperative or partial. Algorithms 4.1, 4.3, and 4.4 describe the implemented retrieval, validation, and scheduler procedures. Algorithms 4.5, 4.6, 4.7, 4.8, 4.9, and 4.10 present the lifecycle, context-paging, cooperative execution, semantic planning, DAG, and global scheduling specifications in that order. The separate priority-aging/time-slice model previously shown as Figure 4.7 has been removed; the implemented priority equation and scheduler procedure are documented earlier in this chapter.

Algorithm 4.5 Agent Lifecycle Management

Require: $agent_id$, $agent_type$, $task_id$
Ensure: Managed agent runtime
1: $runtime_store[agent_id] = (id, type, task_id, run_fn, status)$
2: $status \in \{\text{CREATED}, \text{RUNNING}, \text{COMPLETED}, \text{FAILED}\}$
3: **function** CREATEAGENT($agent_id, agent_type, task_id$)
4: **if** $agent_id \in runtime_store$ **then**
5: **raise** "Agent already exists"
6: **end if**
7: $run_fn \leftarrow$ GETAGENTRUNFUNCTION($agent_type$)
8: $runtime_store[agent_id] \leftarrow (agent_id, agent_type, task_id, run_fn, \text{CREATED})$
9: **return** $runtime_store[agent_id]$
10: **end function**
11: **function** SETAGENTSTATUS($agent_id, status$)
12: $runtime_store[agent_id].status \leftarrow status$
13: **end function**
14: **function** DESTROYAGENT($agent_id$)
15: DELETE(runtime_store[agent_id])
16: **end function**

Algorithm 4.6 Virtual Memory Context Paging

Require: T_{id}, query q, messages $M = \langle m_1, \ldots, m_N \rangle$, window W, threshold τ, top-K limit K, embedding function Φ
Ensure: C_{final}, the context supplied to the LLM

$C_{final} \leftarrow [\text{SystemMessage}(\text{SYSTEM_PROMPT})]$; $P_{in} \leftarrow \emptyset$
if $N > W$ **then**
  $P_{overflow} \leftarrow M[1 : N - W]$
  for all $m_i \in P_{overflow}$ **do**
    $page_key \leftarrow T_{id} \| \texttt{_msg_} \| i$
    if not EXISTS($page_key$) **then**
      $v_i \leftarrow \Phi(m_i.content)$
      $metadata \leftarrow \{T_{id}, m_i.role, m_i.timestamp, i\}$
      VECTORSTORE.INSERT($page_key, v_i, m_i.content, metadata$)
      EMITEVENT(`context.PAGE_OUT`, T_{id}, i)
    end if
  end for
  $M_{active} \leftarrow M[N - W + 1 : N]$
else
  $M_{active} \leftarrow M$
end if
if $N > W$ **and** $\text{Trim}(q) \neq \emptyset$ **then**
  $v_q \leftarrow \Phi(q)$; $Candidates \leftarrow$ VECTORSTORE.SEARCH(T_{id}, v_q, K)
  for all $p \in Candidates$ **do**
    $s(p) \leftarrow \dfrac{v_q \cdot v_p}{\|v_q\| \|v_p\|}$
    if $s(p) \geq \tau$ **then**
      $P_{in} \leftarrow P_{in} \cup \{p\}$
      EMITEVENT(`context.PAGE_IN`, $T_{id}, s(p), p.text$)
    end if
  end for
end if
if $P_{in} \neq \emptyset$ **then**
  $R \leftarrow$ FORMATPAGEDMEMORY(P_{in})
  $C_{final} \leftarrow C_{final} \cup \{\text{SystemMessage}(R)\}$
end if
for all $m \in M_{active}$ **do**
  if $m.role =$ `human` **then**
    C_{final}.APPEND(HumanMessage($m.content$))
  else if $m.role =$ `ai` **then**
    C_{final}.APPEND(AIMessage($m.content$))
  end if
end for
return C_{final}

Algorithm 4.7 Cooperative Time-Sliced Agent Execution

Require: Agent A^* and time quantum T_q
Ensure: Status in $\{\text{COMPLETED}, \text{WAITING}, \text{PREEMPTED}, \text{FAILED}\}$

```
1:  procedure EXECUTEAGENT(A*, T_q)
2:      if A*.execution_context exists then
3:          runtime ← RESTOREEXECUTIONCONTEXT(A*)
4:      else
5:          runtime ← INITIALIZEAGENTRUNTIME(A*)
6:      end if
7:      start_time ← GETCURRENTTIME; SETAGENTSTATUS(A*.id, RUNNING)
8:      while true do
9:          if AGENTEXECUTIONFAILED(runtime) then
10:             SAVEEXECUTIONCONTEXT(A*, runtime); return FAILED
11:         end if
12:         if AGENTISWAITING(runtime) then
13:             SAVEEXECUTIONCONTEXT(A*, runtime); return WAITING
14:         end if
15:         if GETCURRENTTIME − start_time ≥ T_q then
16:             SAVEEXECUTIONCONTEXT(A*, runtime); SUSPENDAGENTEXECUTION(runtime); return PREEMPTED
17:         end if
18:         result ← EXECUTENEXTSTEP(runtime)
19:         if result = COMPLETED then
20:             A*.result ← GETEXECUTIONRESULT(runtime); DESTROYEXECUTIONCONTEXT(runtime); return COMPLETED
21:         else if result = WAITING then
22:             SAVEEXECUTIONCONTEXT(A*, runtime); return WAITING
23:         else if result = FAILED then
24:             SAVEEXECUTIONCONTEXT(A*, runtime); return FAILED
25:         end if
26:     end while
27: end procedure
```

Algorithm 4.8 Semantic Dependency-Aware Agent Planning

Require: User task q, registry A, top-K limit K, planner model L
Ensure: Valid execution DAG $G = (V, E)$

if capability index is not initialized **then**
    for all $a_i \in A$ **do**
        $d_i \leftarrow$ capability description; $e_i \leftarrow \text{Embed}(d_i)$; store (a_i, d_i, e_i)
    end for
end if
$e_q \leftarrow \text{Embed}(q)$; $C \leftarrow \text{SelectTopKAgents}(A, \text{CosineSimilarity}(e_q, e_i), K)$
$P \leftarrow \text{BuildPlannerPrompt}(q, C)$; $R \leftarrow \text{LanguageModelGenerateJSON}(L, P)$
if R contains valid DAG steps **then**
    $S \leftarrow R.steps$
else
    $S \leftarrow \text{ConvertAgentListToSequentialSteps}(R.agents)$
end if
$S \leftarrow \text{RemoveUnknownAgents}(S, A)$
$S \leftarrow \text{RemoveAgentsNotInCandidates}(S, C)$
$S \leftarrow \text{RemoveDuplicateStepIDs}(S)$; $S \leftarrow \text{RemoveInvalidDependencies}(S)$
$S \leftarrow \text{ValidateAcyclicity}(S)$
if S contains a cycle **then**
    $S \leftarrow \text{ConvertAgentListToSequentialSteps}(R.agents)$
end if
$G \leftarrow \text{BuildGraph}(S)$; $M \leftarrow \text{GenerateMermaidDiagram}(G)$
$\text{Persist}(G, M)$; $\text{EmitPlanningEvent}(G, M)$
return G

Algorithm 4.9 Dependency-Aware Agent DAG Execution

Require: Execution DAG $G = (V, E)$ and task state
Ensure: Results produced by all executed agents

$completed \leftarrow \emptyset$
while $completed \neq V$ **do**
    $ready \leftarrow \emptyset$
    for all $v \in V$ **do**
        if $v \notin completed$ **and** Dependencies$(v) \subseteq completed$ **then**
            $ready \leftarrow ready \cup \{v\}$
        end if
    end for
    if $ready = \emptyset$ **then**
        raise InvalidPlanCycle
    end if
    $v \leftarrow$ SchedulerSelect$(ready)$; $result \leftarrow$ ExecuteAgent$(v.agent)$
    StoreResult$(v, result)$; $completed \leftarrow completed \cup \{v\}$
end while
return all results

Algorithm 4.10 Global Priority-Aging Cooperative Agent Scheduler

Require: Redis ready queue Q, aging factor α, maximum priority P_{max}, quantum T_q, resource manager R
Ensure: Scheduled agent execution

while AgentOS is active **do**
  Receive `AGENT_READY` events and insert agents into Q
  if $Q = \emptyset$ **then**
    Wait for the next scheduler event; **continue**
  end if
  $t \leftarrow \text{CurrentTime}()$
  for all $A_i \in Q$ **do**
    $W_i \leftarrow t - A_i.ready_time$; $P_i^{eff} \leftarrow \min(P_i^{base} + \alpha W_i, P_{max})$
  end for
  $A^* \leftarrow \arg\max_{A_i \in Q} P_i^{eff}$; $auth \leftarrow R.\text{request}(A^*)$
  if $auth =$ DENIED **then**
    if denial is temporary **then**
      Mark A^* WAITING and reinsert into Q
    else
      Mark A^* FAILED and publish `AGENT_FAILED`
    end if
    continue
  end if
  Remove A^* from Q; mark RUNNING; publish `AGENT_RUNNING`
  $r \leftarrow \text{Dispatch}(A^*, T_q)$
  if $r =$ COMPLETED **then**
    Mark COMPLETED; publish `AGENT_COMPLETED`
  else if $r =$ WAITING **then**
    Mark WAITING
  else if $r =$ PREEMPTED **then**
    Mark READY, update *ready_time*, reinsert into Q, publish `AGENT_PREEMPTED`
  else
    Mark FAILED; publish `AGENT_FAILED`
  end if
  ReleaseResources(A^*); RecordMetrics(A^*, r)
end while

CHAPTER 5

RESULTS AND PERFORMANCE ANALYSIS

This chapter reports the evaluation of three AgentOS subsystems: the context-paging engine, the model-backed planner, and the global agent scheduler. The results are presented in a consistent order. First, the reporting protocol defines the datasets, comparison conditions, and metrics. The chapter then evaluates context paging, planner tool selection, and scheduler behaviour in sequence. The final section consolidates the findings and states the limitations of the measurements.

5.1 REPORTING PROTOCOL

The evaluations use fixed task or question sets, documented configurations, and reproducible scoring rules. For each experiment, the expected evidence, tool set, or scheduler outcome was defined before the trial. Raw trial records were retained so that aggregate values could be recomputed. Percentages are pooled over the reported trials; latency values are reported using the aggregate statistic available for each run, with percentile values used for queueing measurements.

The context-paging experiments use deterministic synthetic conversations and a fixed pilot subset of the LoCoMo long-term conversational-memory dataset and benchmark [30]. The planner experiment uses the AgentOS function catalogue and a fixed 50-trial task-selection run. The scheduler experiment uses deterministic smoke scenarios and seeded synthetic workloads. These are project-level implementation and reliability evaluations, not claims of production capacity or general performance.

5.2 CONTEXT-PAGING ENGINE

The context engine was evaluated using two complementary datasets. The first is a deterministic synthetic smoke set designed to verify the paging pipeline, telemetry, and comparison harness. The second is a pilot evaluation derived from the LoCoMo benchmark. The configurations compare complete-context access, recent-message truncation, vector retrieval, and AgentOS paging.

The results are retrieval-oriented. A successful retrieval trial means that the annotated evidence required to answer the question is visible in the active context or is returned by the paging operation. This measure is intentionally separate from end-to-end language-model answer accuracy, which requires a fixed answer model and judge and was not enabled for these runs.

5.2.1 Evaluation protocol

All configurations used the same local ChromaDB embedding function, active context window, question text, conversation history, and candidate budget within each dataset. The full-context configuration is a quality control: it receives the complete conversation and therefore does not perform retrieval. Recent truncation retains only the active window. Vector RAG performs direct semantic retrieval, while AgentOS paging combines a larger semantic candidate pool with one-message neighbourhood expansion and a lightweight lexical reranking stage.

The synthetic run expanded five seed scenarios into a fixed local smoke set and executed 400 configuration trials. The external run used a fixed pilot subset of the official LoCoMo questions and executed 800 configuration trials. The threshold sweep was retained as a sensitivity analysis rather than as

evidence that a threshold was universally optimal. Page-in and page-out latencies, embedding-cache behaviour, token estimates, and raw trial records were recorded for every configuration.

5.2.2 Synthetic smoke evaluation

The synthetic evaluation checks whether the implementation can page out older messages, retrieve relevant evidence, handle an absent fact through abstention, and reduce the active prompt size. Table 5.1 reports the principal aggregate values. The full-context result is shown as a control and must not be interpreted as a paging result.

Table 5.1: Synthetic context-paging evaluation

Configuration	Success (%)	Recall@10 (%)	MRR@10 (%)	Token reduction (%)
Full context control	100.0	–	–	0.0
Recent truncation	20.0	–	–	31.8
Vector RAG	98.0	97.5	84.2	5.0
AgentOS paging ($\tau = 0.40$)	98.0	97.5	86.3	21.0

The synthetic results confirm that paging is operational and that the active context is reduced relative to complete-context access. Vector RAG provides the highest retrieval score on this deliberately simple dataset, whereas AgentOS uses fewer active-context tokens and records separate page-out and page-in telemetry. The lower score for recent truncation demonstrates that retaining only the newest messages is insufficient when the answer depends on an older fact.

5.2.3 LoCoMo pilot evaluation

The LoCoMo pilot introduces longer conversations, natural-language questions, and human-provided evidence annotations. Table 5.2 shows the retrieval results. The reported AgentOS operating point uses the best observed region of the threshold sweep ($\tau = 0.20$–0.30); the result is presented as a pilot observation and not as a threshold selected from a held-out development set.

Table 5.2: LoCoMo pilot context-retrieval evaluation

Configuration	Success (%)	Recall@10 (%)	MRR@10 (%)	Token reduction (%)
Full context control	100.0	–	–	0.0
Recent truncation	6.0	–	–	95.0
Vector RAG	22.0	18.2	7.1	93.1
AgentOS paging ($\tau = 0.20$–0.30)	35.0	32.8	13.7	92.9

On the more difficult conversational data, AgentOS paging improves retrieval success by 13 percentage points over the vector-RAG baseline and improves Recall@10 by 14.6 percentage points. The improvement is obtained while maintaining approximately the same 93% reduction in active-context tokens. This indicates that the neighbourhood-aware paging policy is better suited to multi-turn evidence than retrieving isolated messages alone. The absolute scores also show that long-term memory retrieval remains a significant open problem; the result should not be described as perfect question answering.

5.2.4 Graphical analysis

Figure 5.1 compares the two datasets. The complete-context control is intentionally strong, while recent truncation illustrates the cost of aggressive context reduction. The external pilot produces a more challenging separation

between the two retrieval systems than the synthetic fixture, which is evidence that the external benchmark is exercising genuine retrieval difficulty.

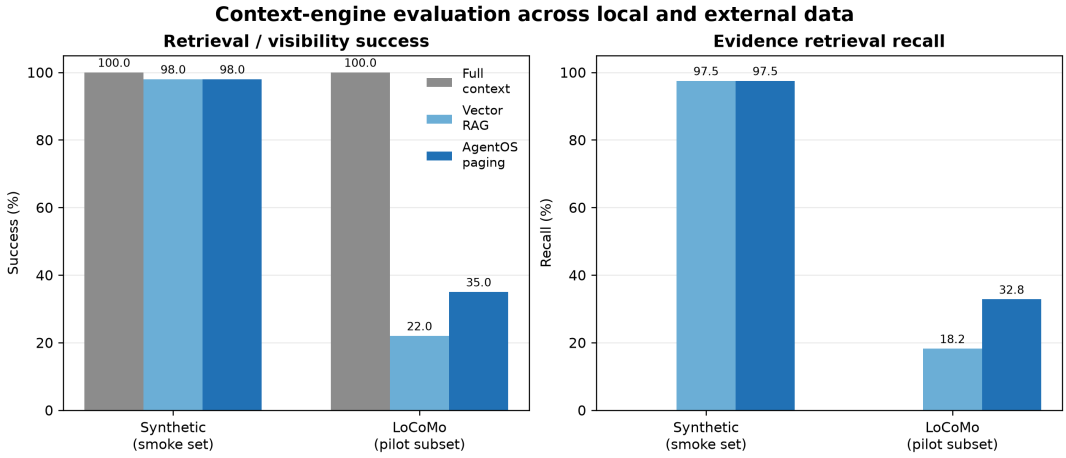


Figure 5.1: Success and evidence-recall comparison for the synthetic and LoCoMo pilot evaluations

Figure 5.2 presents the threshold sensitivity study. The synthetic set favours a higher threshold because its facts are short and lexically distinctive. The LoCoMo pilot favours the lower threshold region, where semantically related but differently worded historical turns are less likely to be rejected. This supports using threshold tuning as a documented validation step rather than treating the default threshold as a universal constant.

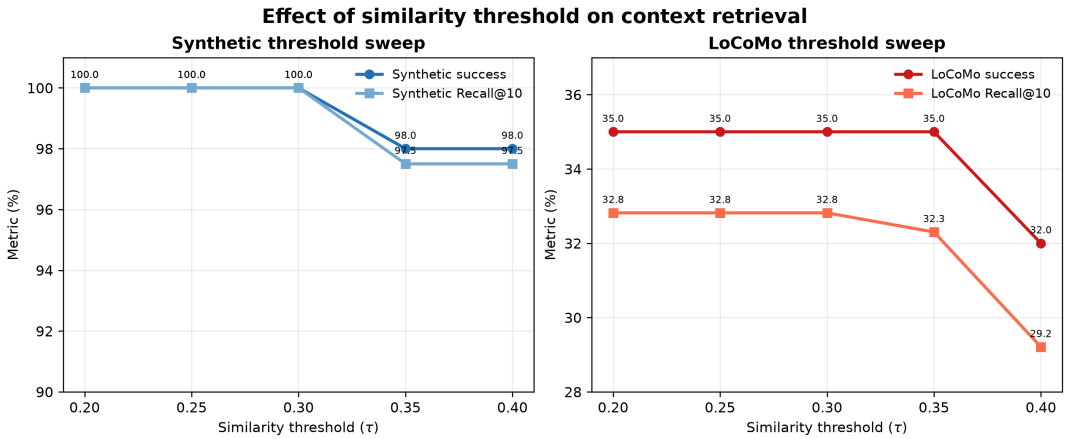


Figure 5.2: Similarity-threshold sensitivity for context retrieval

5.3 PLANNER AND TOOL SELECTION

The planner evaluation measures whether the model selects the registered AgentOS functions required by a task. The model endpoint was Groq using `openai/gpt-oss-120b`. Each registered agent was exposed as an OpenAI-compatible function. The same catalogue and routing prompt were used for every trial, and required tool-call mode was enabled because every task required at least one registered agent.

Exact tool-set accuracy is the fraction of trials for which the selected agent functions exactly matched the expected set. Required-tool recall is the fraction of trials in which every expected function was selected. The end-to-end measures count a failed model tool call as an incorrect outcome. Provider errors and model tool-call failures are recorded separately.

Table 5.3 summarises the 50-trial run. Thirty-eight trials produced the expected tool set, while twelve trials failed at the tool-call stage. No provider errors were recorded. The aggregate latency, computed from category-level means weighted by trial count, was 1003.36 ms.

Table 5.3: AgentOS tool-selection adapter results

Measure	Result
Total trials	50
Successful API trials	50
Correct tool sets	38/50 (76.0%)
End-to-end exact tool-set accuracy	38/50 (76.0%)
End-to-end required-tool recall	38/50 (76.0%)
Model tool-call failures	12 (24.0%)
Provider or transport errors	0
Weighted mean latency	1003.36 ms

The category results indicate strong performance in code execution, with 100% exact-match accuracy, followed by date and time, weather, system

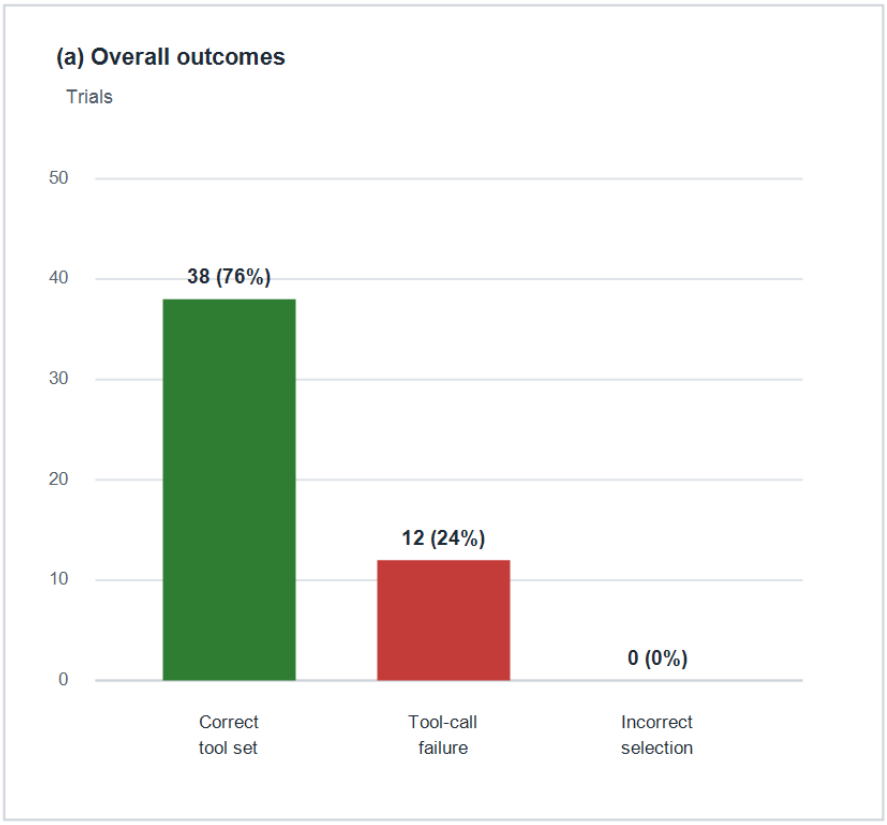


Figure 5.3: Overall outcomes for the 50-trial AgentOS tool-selection evaluation

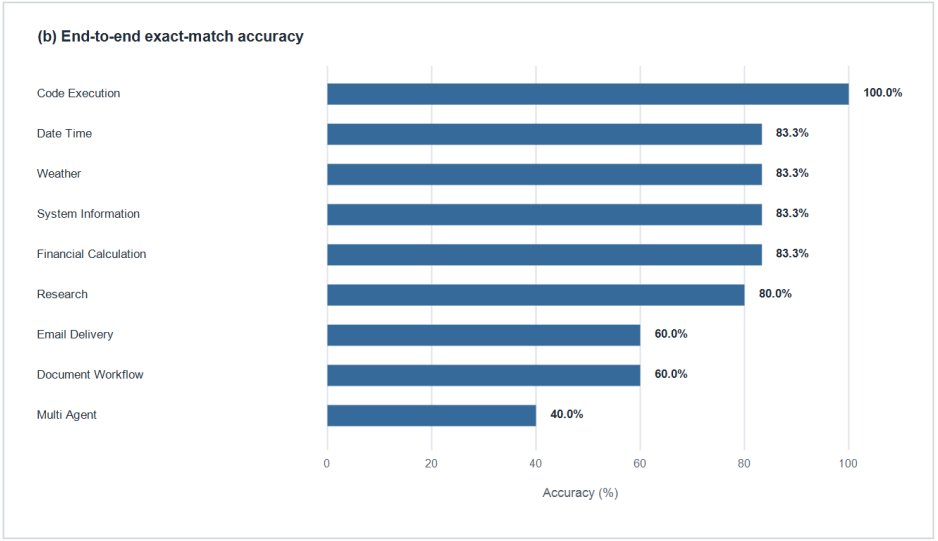


Figure 5.4: End-to-end exact-match accuracy by task category

information, and financial calculation at 83.3% each. Research achieved 80.0%. Email delivery and document workflow achieved 60.0% each, while multi-agent coordination was the weakest category at 40.0%. These results suggest that the principal limitation is tool-call compliance and routing complexity rather than provider availability.

Figures 5.3 and 5.4 visualise the overall outcome distribution and the category-level exact-match results, respectively.

5.4 AGENT SCHEDULER

The scheduler evaluation consists of a deterministic smoke suite and a controlled workload benchmark. The smoke suite executed 21 trials covering concurrency and task limits, cooperative preemption, cumulative token budgets, oversized jobs, priority ordering, quota retry, and runner failure; all 21 completed successfully.

The workload benchmark compared the implemented priority-aging policy with an admission-aware FIFO policy using the same seeded workloads, service-time distributions, and priority values. Each policy and workload profile was repeated ten times, producing 40 trials in total. Persistence was stubbed, so the measurements capture in-process scheduling and asynchronous runner behaviour rather than Redis or MongoDB network overhead. To preserve readability, the results are separated by workload profile: steady-state results are reported in Table 5.4, and queue-contention results are reported in Table 5.5.

Table 5.4: Priority-aging scheduler versus FIFO under steady-state workload

Policy	Throughput (jobs/s)	Queue P_{95} (ms)	High-priority P_{95} (ms)	End-to-end P_{95} (ms)
Priority aging	254.27	218.06	65.18	233.44
FIFO	258.00	214.48	206.00	229.96

Table 5.4 shows that priority aging reduces the high-priority queue-wait percentile under steady-state load, while overall throughput remains close to FIFO. The corresponding queue-contention results are presented separately in Table 5.5.

Table 5.5: Priority-aging scheduler versus FIFO under queue-contention workload

Policy	Throughput (jobs/s)	Queue P_{95} (ms)	High-priority P_{95} (ms)	End-to-end P_{95} (ms)
Priority aging	85.92	511.92	101.41	537.25
FIFO	85.84	508.35	459.55	536.64

Under contention, the separated results make the main scheduling trade-off explicit: priority aging substantially lowers high-priority waiting time while maintaining nearly identical throughput and end-to-end latency.

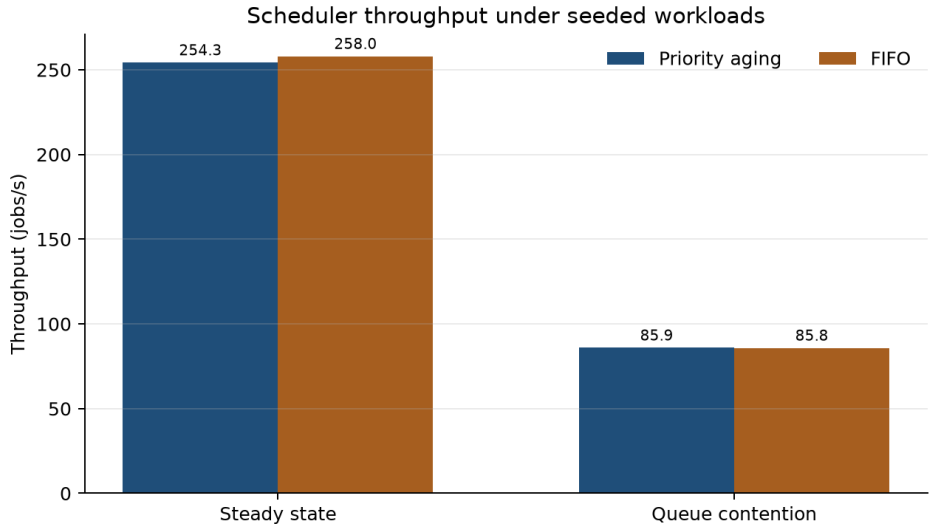


Figure 5.5: Measured scheduler throughput under steady-state and queue-contention workloads

Priority aging reduced high-priority queue-wait P_{95} by 140.82 ms in steady state and by 358.14 ms under contention, corresponding to approximate reductions of 68% and 78% relative to FIFO. Throughput remained effectively unchanged under contention (+0.10%) and decreased by 1.45% in steady state. Overall queue-wait P_{95} increased by approximately 3.6 ms in both profiles because the policy protects urgent work by allowing lower-priority jobs to wait

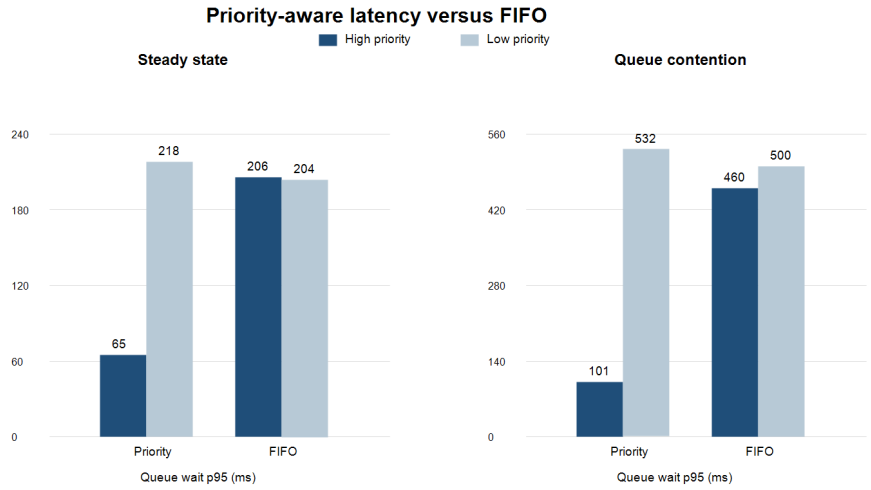


Figure 5.6: High- and low-priority queue-wait P_{95} under priority aging and FIFO

longer. No concurrency-limit violations or cross-task completion imbalance were observed. These measurements should be interpreted as a controlled same-host comparison, not as a production-capacity claim.

Figures 5.5 and 5.6 provide the corresponding throughput and queue-latency visualisations.

5.5 INTEGRATED INTERPRETATION AND LIMITATIONS

The context engine demonstrates measurable active-context reduction and improved retrieval over direct vector RAG on the LoCoMo pilot, although the synthetic smoke set remains easier than the external conversational data. The planner achieves 76.0% end-to-end exact tool-set accuracy and required-tool recall across 50 trials; its main failure mode is missing required tool calls rather than provider unavailability. The scheduler preserves throughput while substantially reducing high-priority queue waiting under contention.

These results have important limitations. The context experiments

use a deterministic synthetic set and a fixed LoCoMo pilot rather than the complete benchmark; threshold selection is exploratory, and no answer-model judging or confidence intervals were enabled. The planner evaluation uses an AgentOS-specific task set and should not be generalized beyond that scope. The scheduler benchmark uses deterministic service times and stubbed persistence, so it is not a production-capacity claim. Accordingly, the findings establish implementation behaviour under the stated conditions, not universal superiority over alternative systems. Future work should evaluate held-out planner tasks and thresholds, add answer judging, replace stubbed persistence, and use production-like scheduler traces.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project presented AgentOS, a modular runtime for multi-agent applications in which planning, semantic agent selection, DAG execution, context management, scheduling, resource admission, sandboxed code execution, provider routing, and frontend observability are treated as shared system services. The implemented system connects the Next.js interface, FastAPI gateway, Python kernel, planner, Redis event path, MongoDB task history, ChromaDB retrieval, scheduler, resource manager, agent runtime, and frontend activity interface into one observable execution pipeline.

The implementation demonstrates that operating-system-inspired services can be composed around heterogeneous agents without placing scheduling, memory, execution, and observability logic inside each individual agent. Structured plans and dependency validation provide predictable execution order; semantic context paging reduces the prompt context presented to the model; priority aging and resource admission coordinate runnable work; provider routing supplies local/cloud model selection; and bounded subprocess execution provides a controlled path for coding and testing tasks. The frontend activity stream makes these decisions visible to the user.

The evaluation supports three specific conclusions. First, context paging remained effective under the reported conditions: AgentOS achieved 98.0% retrieval success on the synthetic set and 35.0% on the LoCoMo pilot, improving on direct vector retrieval by 13 percentage points on the latter. Second, the planner selected the exact required tool set in 38 of 50 trials, giving an end-to-end accuracy of 76.0%; the remaining failures were model tool-call

failures rather than provider or transport errors. Third, the scheduler preserved comparable throughput while reducing high-priority queue-wait P_{95} by approximately 68% in steady state and 78% under contention relative to FIFO. These results are bounded by the datasets, configurations, and instrumentation described in Chapter 5.

The current implementation also has defined limitations. Scheduler dispatch is cooperative, provider token usage may be unavailable when metadata is omitted, and restart recovery detects interrupted work without restoring every active coroutine. The context and planner evaluations are limited in dataset size and task diversity, while the scheduler benchmark uses deterministic workloads and stubbed persistence. Evaluation results should therefore be interpreted together with their dataset scope, provider conditions, and telemetry coverage rather than as universal claims of system performance.

Future work will first address reliability and recovery. Durable restart resumption should restore incomplete task state without duplicating completed steps, while checkpoints inside long-running agents would strengthen cooperative suspension and continuation. Capability enforcement should also be extended through explicit permission policies and stronger isolation where required.

Further studies should use production-like arrival traces, non-stubbed persistence, cost-aware local/cloud scheduling, and changing provider conditions. The context and planner evaluations should include held-out conversations, paraphrased tasks, answer-level judging, and larger repeated trials. Broader tool-use scenarios and autonomous event triggers can then test generalisation beyond the current scope while preserving the separation of planner, scheduler, runtime, persistence, routing, and observability layers.

DECLARATION ON THE USE OF AI TOOLS

Generative artificial intelligence tools were used in this project in a limited and assistive capacity. Their use was restricted to the following purposes:

1. **Literature-survey support:** ResearchRabbit and Elicit, together with AI-assisted language tools, were used to discover, organise, and suggest research papers and other scholarly sources relevant to the themes discussed in the literature survey. The relevance, accuracy, bibliographic details, and applicability of all sources were independently assessed and verified by the authors before inclusion. The Elicit literature-review artifacts used during this process are identified below:

    - Context paging for agent memory: `https://elicit.com/artifact/c6d5e861-97d3-4c92-a45d-26869f67acd0?share_ref=05ce4385-30d3-425c-9a67-419bfe724f39`.
    - Multi-agent priority-aging scheduling: `https://elicit.com/artifact/9dda3c60-9276-4f35-891a-5ff2a38df6d9?share_ref=f4bf66b1-b743-4bcc-919e-b916fd7fae87`.
    - LLM multi-agent planning and workflow optimization: `https://elicit.com/artifact/d99853bf-b74b-4ec6-9564-2fb15eb8e9cf?share_ref=4cb79663-65f3-40b9-88f2-52fe0f3bdc34`.

2. **Language and editorial support:** AI-assisted tools were used to review grammar, spelling, sentence structure, clarity, and academic style in the report. The tools supported editorial refinement only; the

technical content, interpretation of results, methodological decisions, and conclusions were developed, checked, and approved by the authors.

3. **Synthetic test-case support:** AI-assisted tools were used to propose relevant synthetic test cases for evaluating context paging, planner behaviour, and related runtime scenarios. The authors defined the evaluation objectives and acceptance criteria, reviewed the generated candidates, removed unsuitable or duplicated cases, and retained only cases consistent with the approved experimental protocol.

The AI tools were not used to fabricate experimental results, generate unverified references, replace the authors' analysis, or determine the conclusions of the project. The authors retain full responsibility for the originality, accuracy, integrity, and final presentation of the report.

Tools used: OpenAI ChatGPT-4 GO, ResearchRabbit, and Elicit, used under the supervision and review of the authors.

APPENDIX

APPENDIX A
Agent Catalogue and Internal Workflows

This appendix documents the agent implementations in `server/kernel/Agents/agents/`. The diagrams describe the internal responsibility of each agent and its actual tool or model interaction. The planner, scheduler, global execution state, and frontend are intentionally not shown here because they are documented in the system-design chapter. The registered agents and their implemented responsibilities are summarised in Table A.1.

Agent	Implemented responsibility
A1	System information and resource inspection.
A2	Date, time, and timezone queries.
A3	Factual and academic research with sources.
A4	File and document operations with deletion confirmation.
Content Creator	Text drafting, rewriting, and formatting.
Email	Email dispatch and delivery.
Weather	Current weather and forecast retrieval.
Loan EMI	EMI and amortisation calculation.
Crypto PnL	Cryptocurrency trade PnL and ROI reporting.
Code Runner	Python generation, execution, capture, and repair.
Agent Creator	Creation and registration of new specialised agents.
Data Analysis	Structured-data inspection, calculation, and evidence-based findings.
Decision Support	Evidence synthesis, option comparison, and actionable recommendations.
Summarizer	Concise summaries of supplied text, conversations, and agent results.
Translator	Translation and tone-preserving text transformation.
Checklist	Ordered action checklists with dependencies and acceptance checks.

Figure A.1: AgentOS agent catalogue

A.1 Agent A1: System Information Agent

The A1 implementation creates a LangChain agent with the single registered tool `get_system_info`. The tool reports the operating-system name and release, machine architecture, and processor descriptor using Python's `platform` module. The agent therefore provides a compact host identity report; process enumeration and detailed resource monitoring are not implemented by the current tool. The system-information processing logic is shown in Figure A.2.

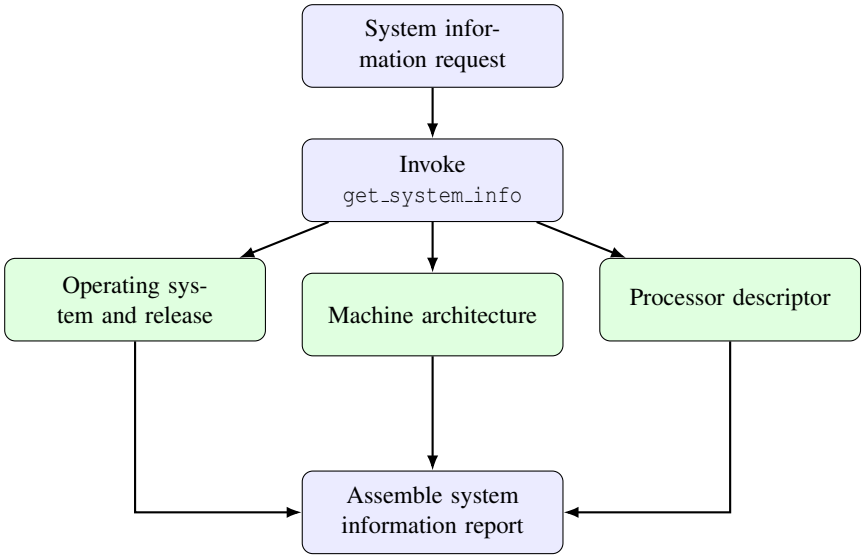


Figure A.2: A1 system-information logic

A.2 Agent A2: Date and Time Agent

The A2 implementation creates a LangChain agent with `get_time`. The tool returns the host's current `datetime.now()` value as a formatted string. It does not currently accept an explicit timezone or location, so its answer represents the runtime host clock. The date-and-time processing logic is shown in Figure A.3.

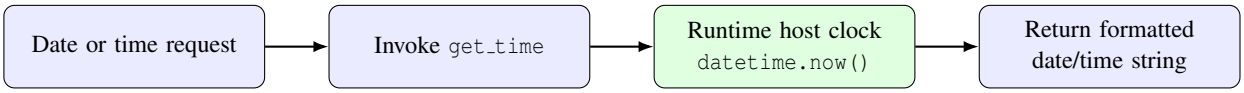


Figure A.3: A2 date-and-time logic

A.3 Agent A3: Research Agent

The A3 implementation uses two tools: `wikipedia_search` calls the Wikipedia search and summary APIs and returns article titles, extracts, and URLs; `trusted_web_search` restricts search results to configured academic and scientific domains, validates result URLs, removes duplicates, and returns source snippets. The agent prompt requires concise evidence and prohibits fabricated citations or URLs. The research and source-verification flow is shown in Figure A.4.

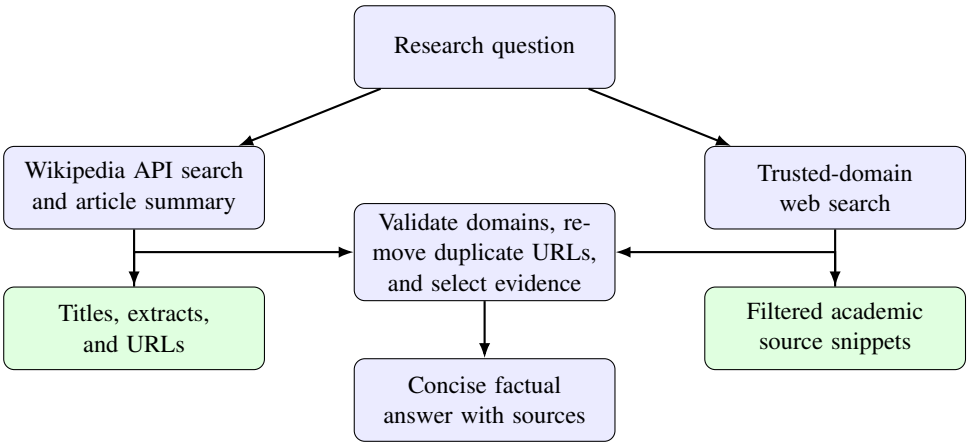


Figure A.4: A3 research and source-verification logic

A.4 Agent A4: File and Document Agent

The A4 implementation exposes file discovery, bounded document reading, TXT/DOCX creation, append and exact replacement, folder creation, rename, move, metadata, directory listing, document comparison, and heuristic classification. PDF and DOCX readers are supported through `pypdf` and `python-docx`. There is deliberately no direct delete tool: `request_delete_confirmation` records a fingerprint in Redis, publishes a frontend confirmation event, and uses a 300-second expiry before the backend performs or cancels the operation. The file-operation and deletion-confirmation flow is shown in Figure A.5.

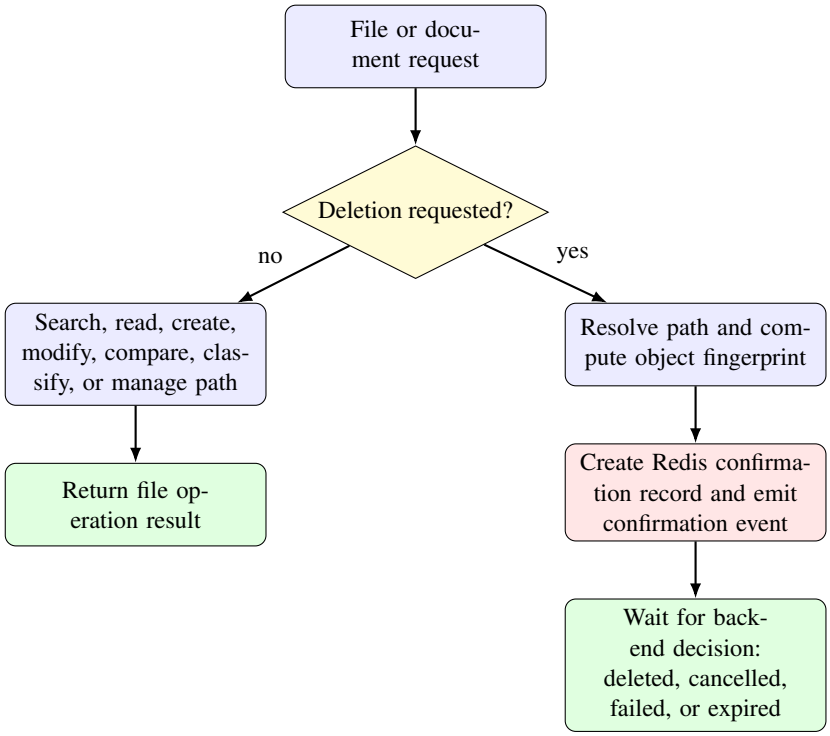


Figure A.5: A4 file-operation and deletion-confirmation logic

A.5 Content Creator Agent

The Content Creator implementation creates a tool-free LangChain agent. It loads the current user name and email for sender context, selects the requested genre, and drafts, rewrites, summarises, or formats text. It is intentionally not an email-delivery component and must not send messages or emit unresolved placeholders. The content-creation flow is shown in Figure A.6.

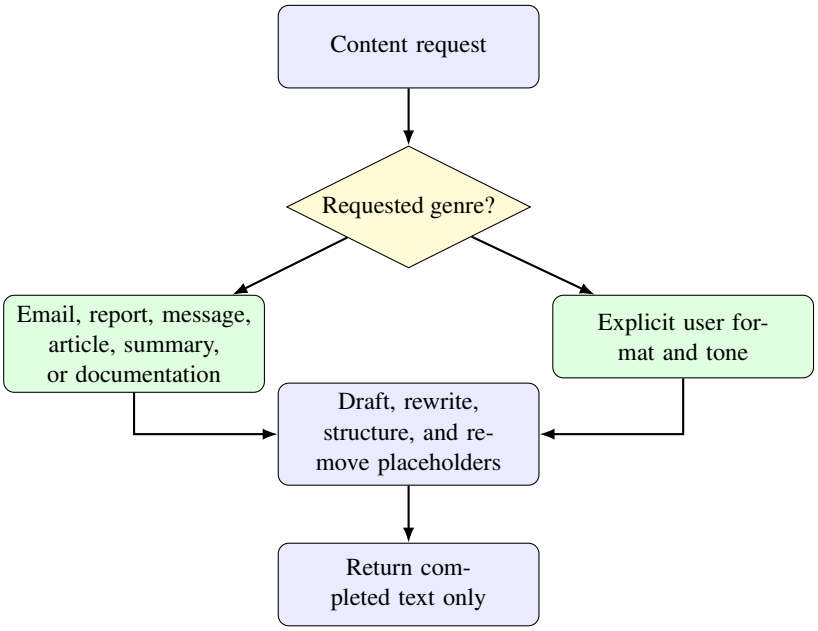


Figure A.6: Content Creator drafting logic

A.6 Email Agent

The Email Agent uses `send_email`. The tool reads SMTP host, port, username, password, TLS, and sender settings from the environment. It sends through SMTP when credentials are available; otherwise it returns an explicit simulation-mode response and does not send network mail. The agent extracts the recipient, subject, and body, removes unresolved placeholders, and returns the tool's delivery status. The email-delivery and simulation flow is shown in Figure A.7.

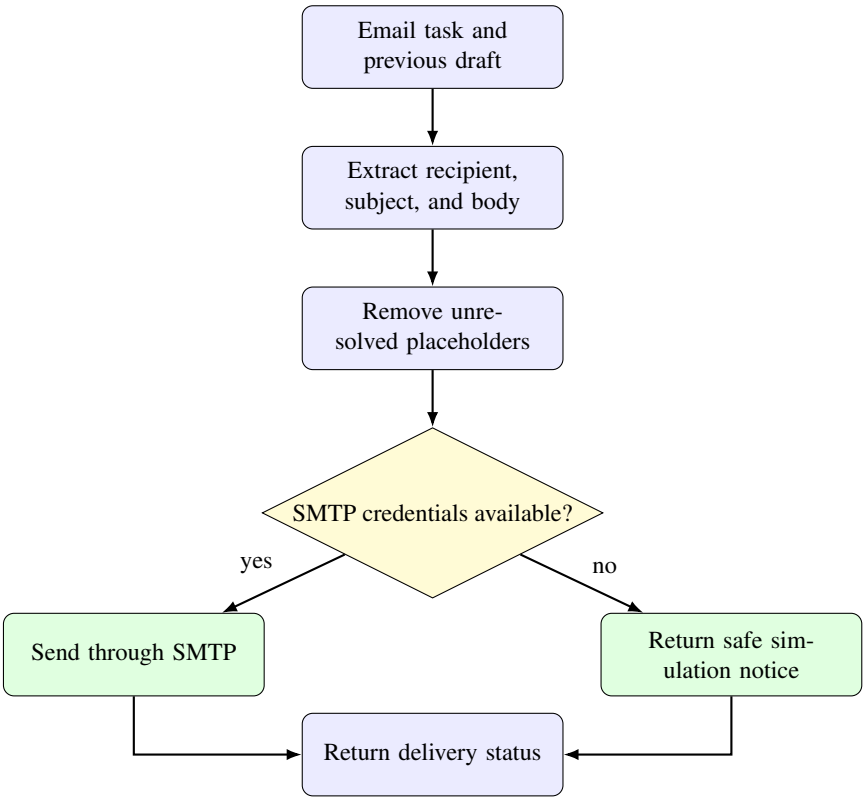


Figure A.7: Email delivery and simulation-mode logic

A.7 Weather Agent

The Weather Agent uses Open-Meteo through `get_current_weather` and `get_weather_forecast`. Each operation geocodes the city first, then requests current or daily forecast data, maps WMO weather codes to readable conditions, and reports Celsius/Fahrenheit temperatures, humidity, wind, and precipitation. Forecast length is bounded to one through seven days.

The weather-retrieval and reporting logic is shown in Figure A.8.

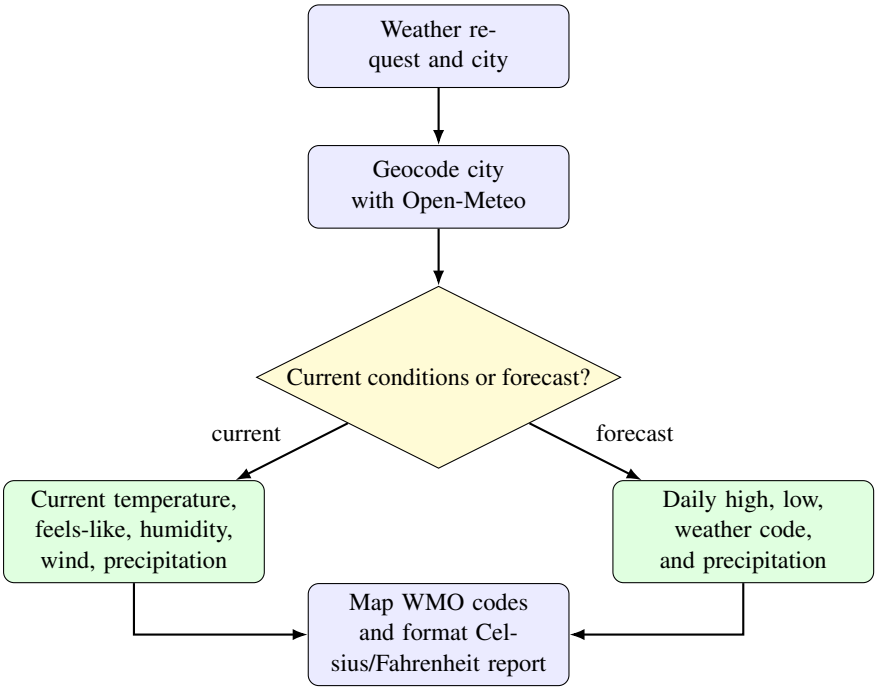


Figure A.8: Weather retrieval and reporting logic

A.8 Loan EMI Calculator Agent

The Loan EMI Agent uses `compute_emi_and_schedule`. It converts the annual percentage rate to a monthly rate, computes the fixed-payment EMI, derives total payment and interest, and produces a monthly schedule for short tenures or an annual amortisation breakdown for longer tenures. The loan-EMI and amortisation logic is shown in Figure A.9.

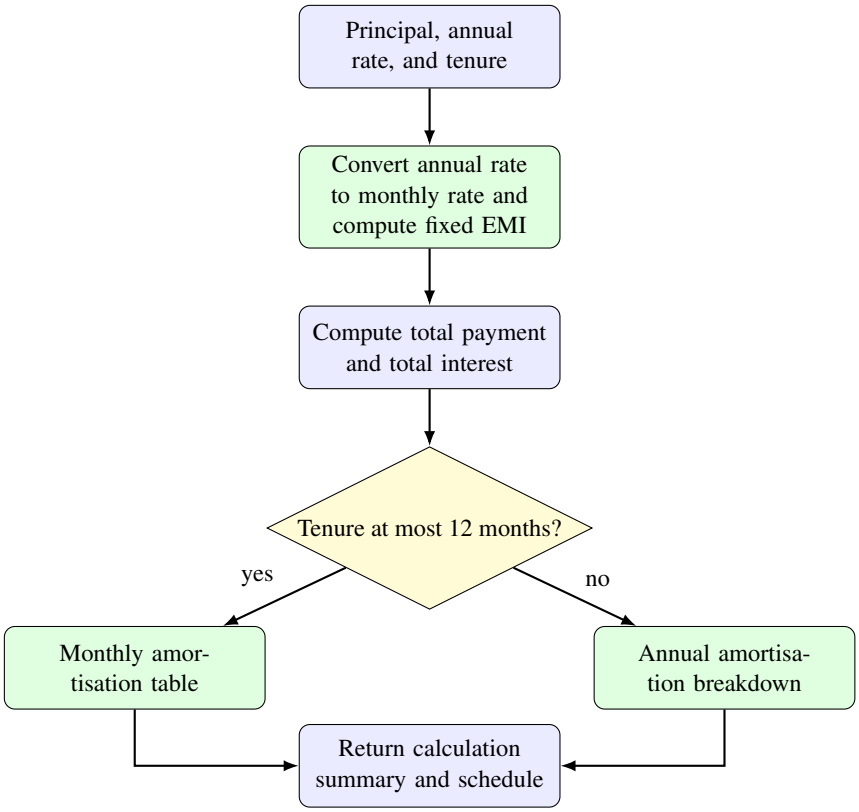


Figure A.9: Loan EMI and amortisation logic

A.9 Crypto PnL Agent

The Crypto PnL Agent uses `calculate_crypto_pnl`. It calculates buy and sell notional values, applies the fee percentage to both sides, derives net profit, and computes ROI relative to total purchase outflow. The cryptocurrency PnL and ROI logic is shown in Figure A.10.

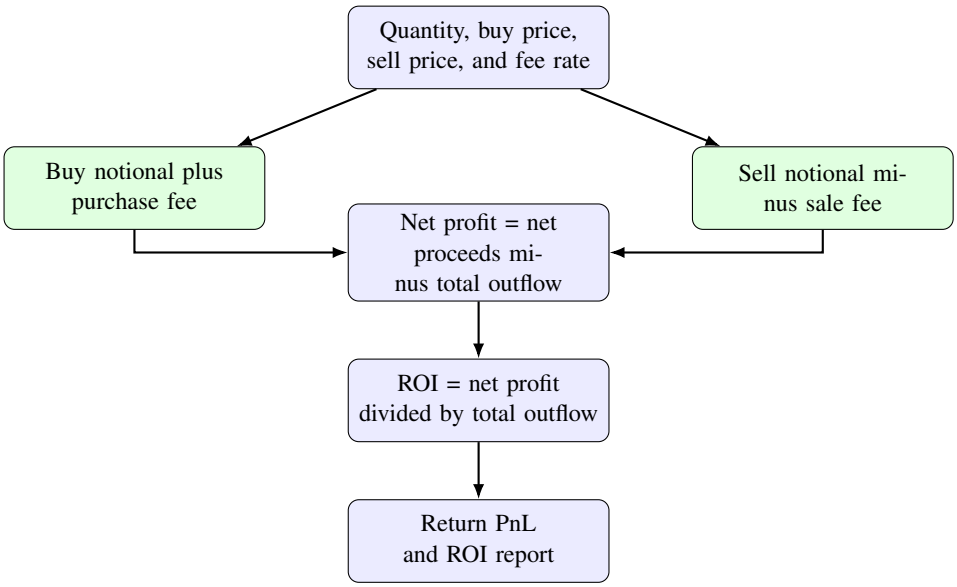


Figure A.10: Cryptocurrency PnL and ROI logic

A.10
Code Runner Agent

The Code Runner Agent uses `build_run_python_code_tool`. It limits source code to 20,000 characters, executes Python with isolated mode and a sanitised environment, bounds runtime to ten seconds, captures at most 64,000 output characters, publishes terminal events, and limits concurrent runs to two. The agent repairs failures for at most three attempts. The code-generation, sandbox-execution, and repair logic is shown in Figure A.11.

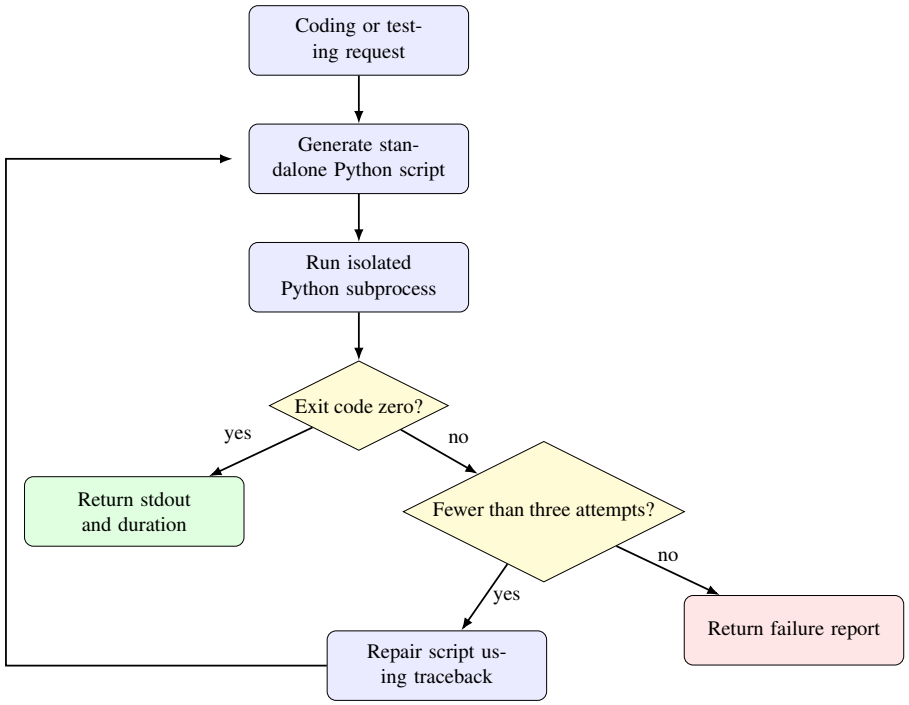


Figure A.11: Code generation, sandbox execution, and repair logic

A.11
Data Analysis Agent

The Data Analysis Agent is a lightweight structured-data agent for CSV, JSON, TSV, and tabular text. It uses the file tools to locate and read the requested source and the bounded Python execution tool for reproducible calculations. The agent is instructed to inspect the source before calculating, state its assumptions, preserve the source data, and report only evidence-supported findings. It does not use a retry loop or modify data by default. The data-analysis flow is shown in Figure A.12.

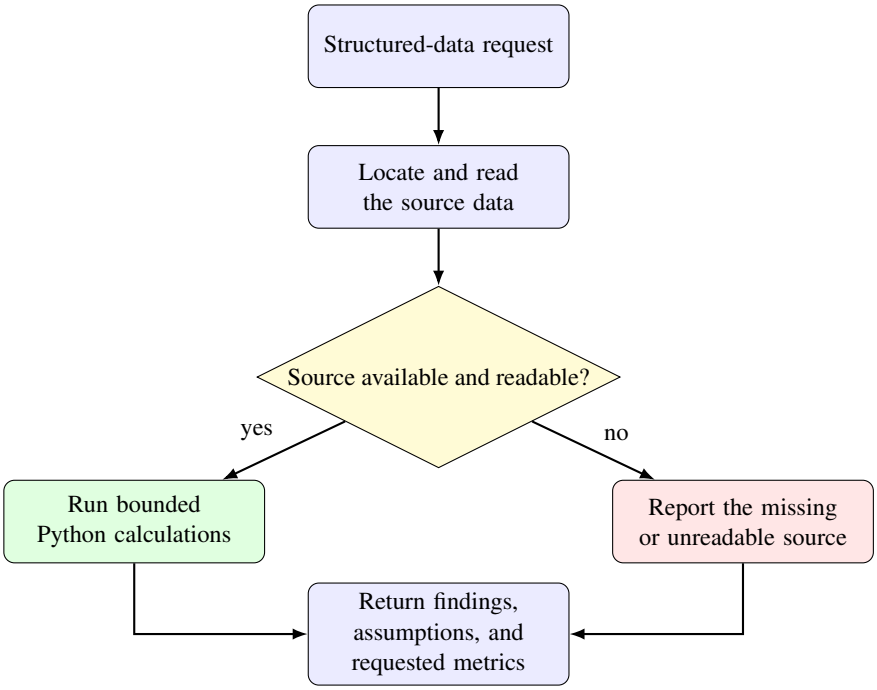


Figure A.12: Data-analysis inspection and calculation logic

A.12
Decision Support Agent

The Decision Support Agent is a lightweight, tool-free synthesis agent. It operates on the task and results supplied by the planner and preceding agents. It compares alternatives, weighs trade-offs, identifies assumptions and uncertainty, and produces a recommendation followed by the most important reasons and next action. It does not retrieve external evidence or invent facts that are not present in the supplied context. The decision-support flow is shown in Figure A.13.

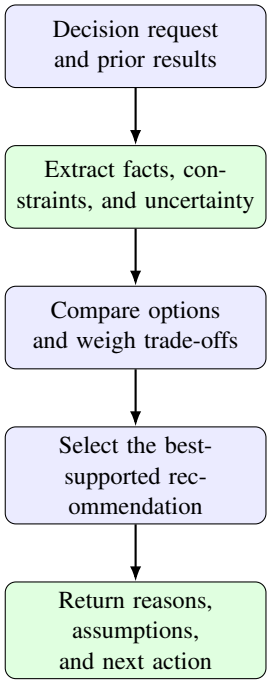


Figure A.13: Decision-support synthesis logic

A.13
Summarizer Agent

The Summarizer Agent is a lightweight, tool-free agent for supplied text, conversation excerpts, documents already loaded into context, and prior agent results. It preserves important facts, decisions, dates, risks, and action items while removing repetition. The prompt explicitly prohibits adding facts that are absent from the supplied material and returns a concise summary. The summarization flow is shown in Figure A.14.

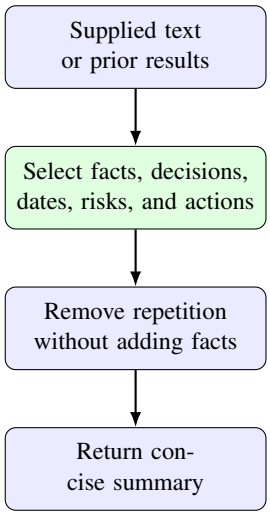


Figure A.14: Summarization and fact-preservation logic

A.14
Translator Agent

The Translator Agent is a lightweight, tool-free agent for translation and tone-preserving rewriting. It preserves meaning, names, numbers, formatting, and placeholders while following the requested target language, audience, and tone. It returns only the transformed text unless the user explicitly asks for an explanation. The translation flow is shown in Figure A.15.

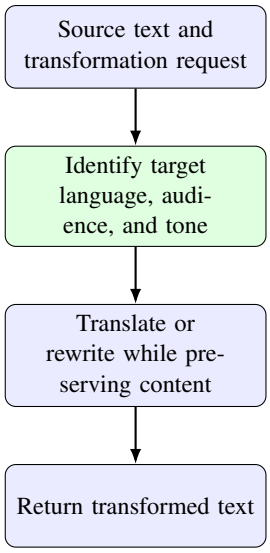


Figure A.15: Translation and tone-preserving transformation logic

A.15
Checklist Agent

The Checklist Agent is a lightweight, tool-free agent that converts a goal, set of requirements, or plan into a short ordered checklist. It includes dependencies, acceptance checks, and likely risks when they are present, while avoiding unnecessary steps or invented requirements. The result is designed to be directly actionable by a user or a downstream agent. The checklist flow is shown in Figure A.16.

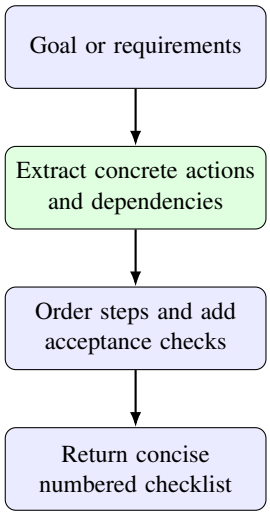


Figure A.16: Checklist construction logic

A.16
Agent Creator

The Agent Creator implementation is responsible for unsupported capabilities. `save_tool_file` parses generated source with the Python AST before writing and registering it. `create_and_register_agent` generates the agent module, updates the agent registry and planner constants, validates the changes, and rolls them back on failure. `run_generated_agent` then loads and executes the new agent. The implementation is present in the source directory but is commented out of the default registry. The dynamic agent-creation and registration logic is shown in Figure A.17.

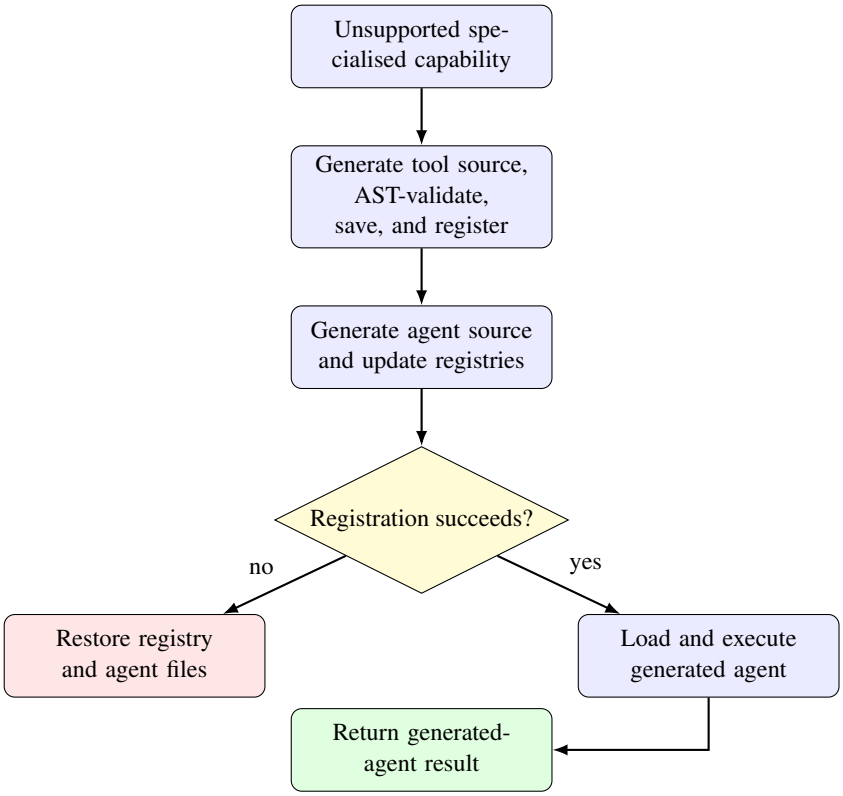


Figure A.17: Dynamic agent creation and registration logic

APPENDIX B

AI-ASSISTED PROMPTS

This appendix records representative prompts corresponding to the limited uses of AI tools declared in the report. The prompts are provided for transparency and reproducibility. They define the intended scope of assistance and require the author to verify all scholarly claims, sources, and editorial changes before incorporation into the report.

B.1 LITERATURE-SURVEY SUPPORT

Prompt B.1: Identification of relevant scholarly sources

> Identify peer-reviewed papers, authoritative conference publications, or official technical documentation relevant to the following literature-survey topics: *context paging for agent memory and optimized planning*. For each suggested source, provide the complete bibliographic details, publication venue and year, DOI or stable URL where available, and a concise explanation of its relevance. Do not invent sources, bibliographic information, results, or quotations. Distinguish clearly between peer-reviewed research, preprints, and technical documentation. Suggest only sources that can be independently verified by the author. Do not write unsupported claims for direct inclusion in the report; instead, identify which claims each verified source can reasonably support and note any important limitations or differences from the present project.

B.2 LANGUAGE AND EDITORIAL SUPPORT

Prompt B.2: Academic language and grammar review

> Review the following report passage for grammar, spelling, punctuation, sentence structure, clarity, consistency, and formal academic style. Preserve the author's technical meaning, numerical results, citations, terminology, section structure, and stated limitations. Do not add new claims, references, experimental results, or interpretations. Identify any sentence whose meaning is ambiguous and explain the issue before proposing a revision. Return: (1) a revised passage with only necessary editorial changes, and (2) a concise list of the substantive edits made.

B.3 SYNTHETIC TEST-CASE SUPPORT

Prompt B.3: Development of candidate synthetic test cases

Using the following approved evaluation objective and system description, propose a set of deterministic synthetic test cases for evaluating *context paging for agent memory, priority-aging scheduling of multi-agent systems, and optimized planning*. Each case should specify a unique identifier, the input scenario, the relevant context or task information, the expected system behaviour, and the measurable success criteria. Ensure that the cases cover both normal operation and meaningful edge conditions, such as missing context, irrelevant retrieved context, repeated information, dependency constraints, and invalid or ambiguous plans. Do not invent implementation capabilities that are not stated in the system description. Do not include private, sensitive, or personally identifying information. Avoid duplicate cases and avoid wording that reveals the expected answer unless it is required for deterministic evaluation. Return the cases in a structured table and clearly identify assumptions.

REFERENCES

[1] K. Mei, X. Zhu, W. Xu, W. Hua, M. Jin, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOS: LLM Agent Operating System," arXiv:2403.16971, 2024. [Online]. Available: https://arxiv.org/abs/2403.16971 [Accessed: Oct. 28, 2026].

[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J.-S. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model Based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024, doi: 10.1007/s11704-024-40231-1.

[3] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering Large Language Models With Optimal Planning Proficiency," arXiv:2304.11477, 2023. [Online]. Available: https://arxiv.org/abs/2304.11477 [Accessed: Oct. 28, 2026].

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155 [Accessed: Oct. 28, 2026].

[5] LangChain, "LangGraph Documentation," 2026. [Online]. Available: https://langchain-ai.github.io/langgraph/ [Accessed: Oct. 6, 2026].

[6] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as Agents," arXiv:2308.03688, 2023. [Online]. Available: https://arxiv.org/abs/2308.03688 [Accessed: Oct. 28, 2026].

[7] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023. [Online]. Available: https://arxiv.org/abs/2310.08560 [Accessed: Oct. 28, 2026].

[8] J. Kang, M. Ji, Z. Zhao, and T. Bai, "Memory OS of AI Agent," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing*, 2025, pp. 25961–25970, doi: 10.18653/v1/2025.emnlp-main.1318.

[9] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," arXiv:2410.10813, 2024. [Online]. Available: https://arxiv.org/abs/2410.10813 [Accessed: Oct. 28, 2026].

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[11] Chroma, "Chroma Documentation: Collections and Configuration," 2026. [Online]. Available: https://docs.trychroma.com/ [Accessed: Oct. 6, 2026].

[12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics: System Demonstrations*, St. Julians, Malta, 2024, pp. 150–158, doi: 10.18653/v1/2024.eacl-demo.16.

[13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proc. 2024 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Language Technologies*, vol. 1, Mexico City, Mexico, 2024, pp. 338–354, doi: 10.18653/v1/2024.naacl-long.20.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learning Representations*, 2023.

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[16] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proc. 36th ACM Symp. User Interface Software and Technology*, 2023, doi: 10.1145/3586183.3606763.

[17] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents With Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[18] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs," arXiv:2307.16789, 2023. [Online]. Available: `https://arxiv.org/abs/2307.16789` [Accessed: Oct. 28, 2026].

[19] L. Yue, K. R. Bhandari, C.-Y. Ko, D. Patel, S. Lin, N. Zhou, J. Gao, P.-Y. Chen, and S. Pan, "From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents," arXiv:2603.22386, 2026, doi: 10.48550/arXiv.2603.22386. [Online]. Available: `https://arxiv.org/abs/2603.22386` [Accessed: Oct. 28, 2026].

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[21] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[22] Linux Kernel Documentation, "Control Group v2," 2026. [Online]. Available: `https://docs.kernel.org/admin-guide/cgroup-v2.html` [Accessed: Oct. 6, 2026].

[23] LangChain, "LangGraph Persistence and Checkpointing," 2026. [Online]. Available: `https://langchain-ai.github.io/langgraphjs/concepts/persistence/` [Accessed: Oct. 6, 2026].

[24] Temporal Technologies, "Temporal Documentation: Durable Execution," 2026. [Online]. Available: `https://docs.temporal.io/` [Accessed: Oct. 6, 2026].

[25] Model Context Protocol, "Model Context Protocol Specification," 2026. [Online]. Available: `https://modelcontextprotocol.io/specification/` [Accessed: Oct. 6, 2026].

[26] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: `https://owasp.org/www-project-top-10-for-large-language-model-applications/` [Accessed: Oct. 6, 2026].

[27] Amazon Web Services, “Firecracker MicroVM Documentation,” 2026. [Online]. Available: https://github.com/firecracker-microvm/firecracker [Accessed: Oct. 6, 2026].

[28] Redis Ltd., “Redis Pub/Sub Documentation,” 2026. [Online]. Available: https://redis.io/docs/latest/develop/interact/pubsub/ [Accessed: Oct. 6, 2026].

[29] MongoDB Inc., “MongoDB Documentation,” 2026. [Online]. Available: https://www.mongodb.com/docs/ [Accessed: Oct. 6, 2026].

[30] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating Very Long-Term Conversational Memory of LLM Agents,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, vol. 1, Bangkok, Thailand, Aug. 2024, pp. 13851–13870, doi: 10.18653/v1/2024.acl-long.747.